人工智能引论

深度学习：序列神经网络

北京郵電大學
Beijing University of Posts and Telecommunications

导认识

导原理

导重点

导兴趣

深度学习常见模型

卷积神经网络模型

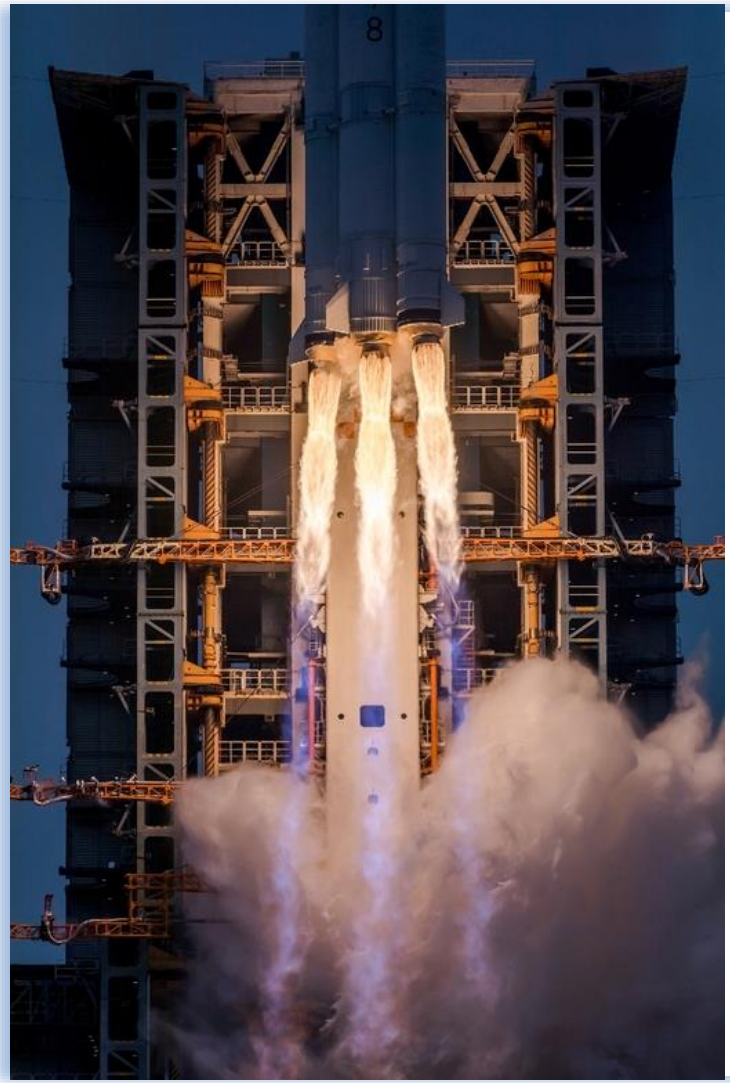
- 卷积操作
- 池化操作
- 经典模型与应用场景

序列模型

- 循环神经网络模型
- 注意力机制
- Transformer 模型

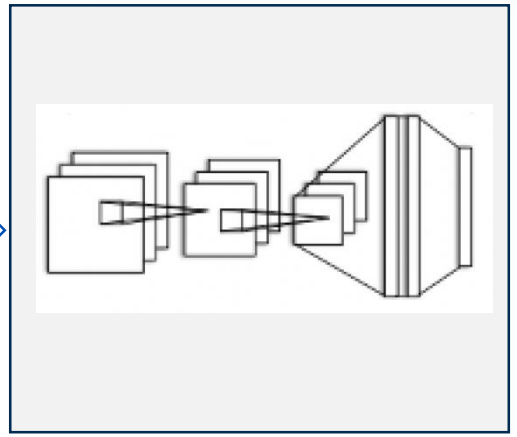
神经网络怎么做到“看图作文”？

北京郵電大學 Beijing University of Posts and Telecommunications



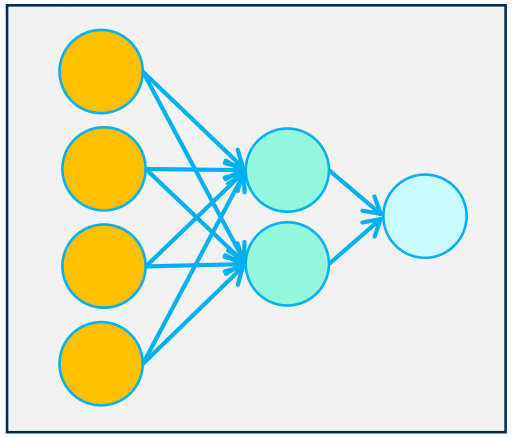
无所不能的深度神经网络

识别图像



卷积神经网络模型

生成文字

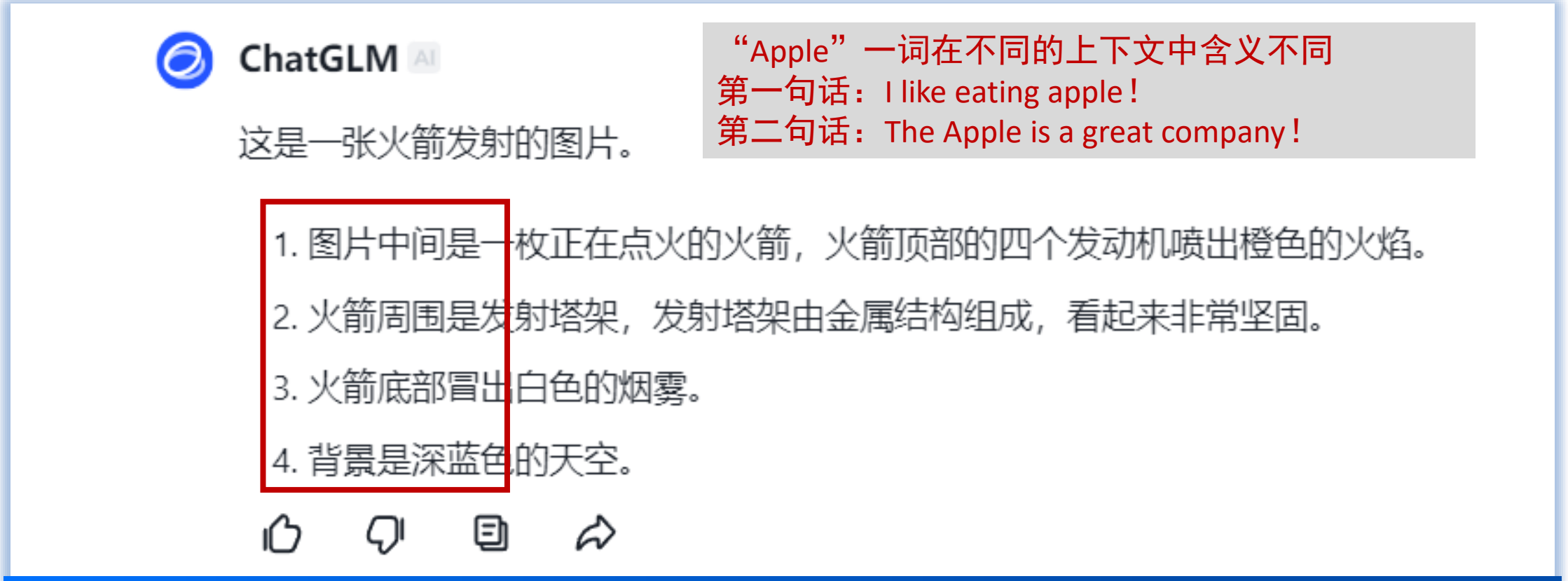


序列模型

这 是 一 张 火 箭 发 射 的 图 片

神经网络怎么做到“看图作文”？

北京郵電大學 Beijing University of Posts and Telecommunications

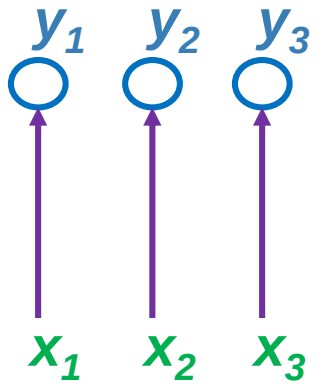


“Apple”一词在不同的上下文中含义不同
第一句话：I like eating apple！
第二句话：The Apple is a great company！

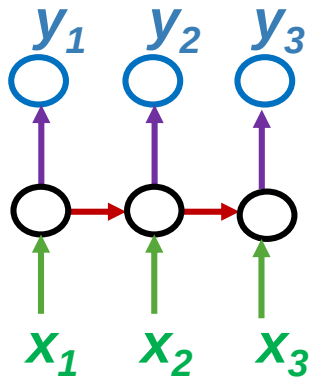
语言是有上下文的，词语间是有先后顺序的
如何让神经网络模型输出符合逻辑的文字？

循环神经网络是考虑顺序的网络结构

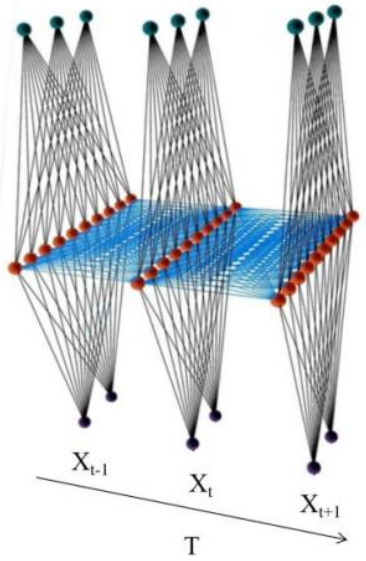
RNN对具有序列特性的数据非常有效，它能挖掘数据中的时序信息以及语义信息，利用了RNN的这种能力，使深度学习模型在解决语音识别、语言模型、机器翻译以及时序分析等NLP领域的问题时有所突破。



不考虑前后关系的
神经网络（FFNN, CNN）

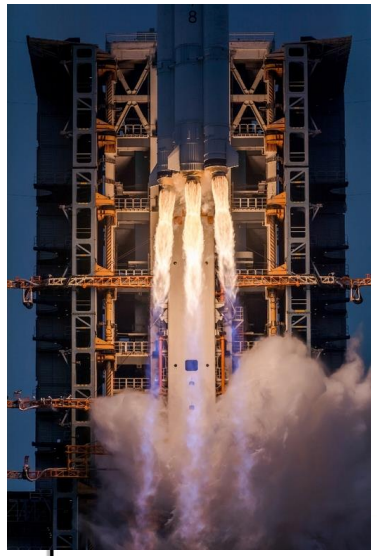


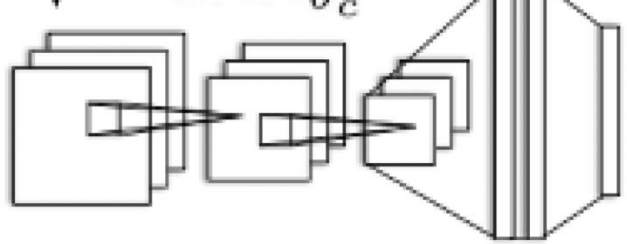
考虑前后关系的
循环神经网络（RNN）

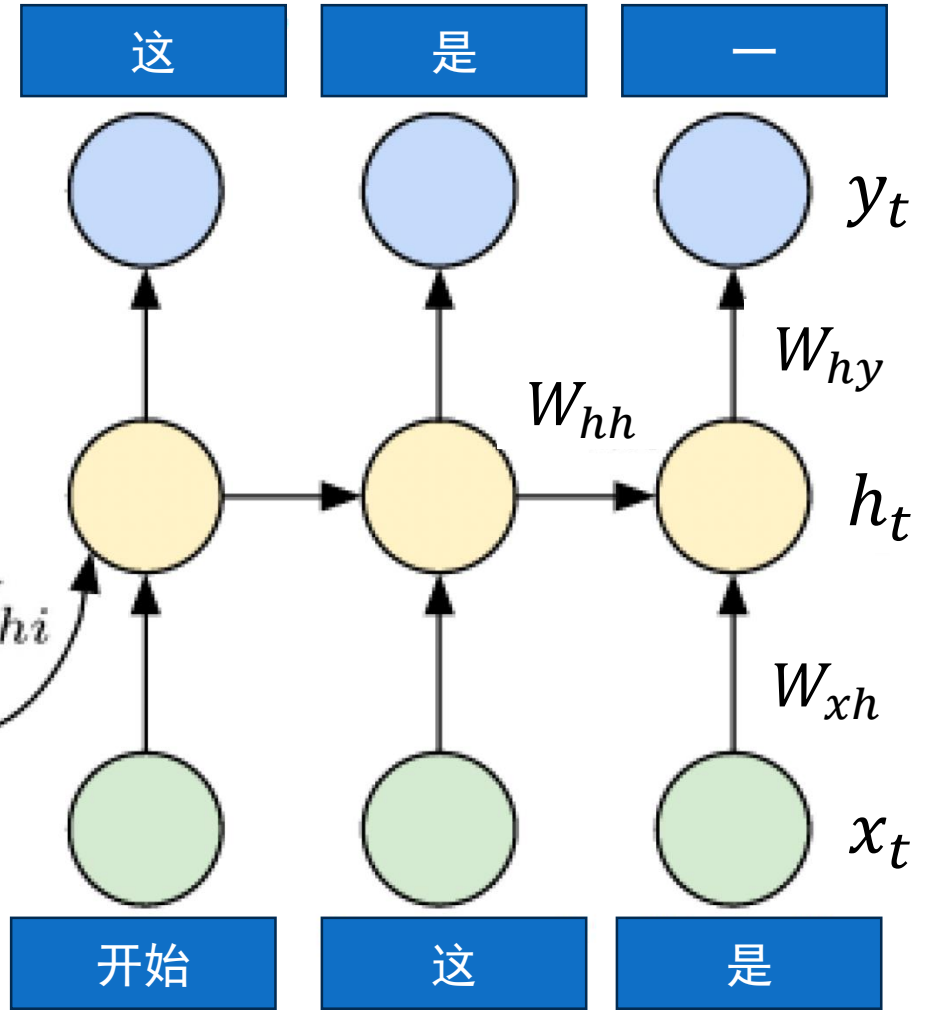


循环神经网络
沿着时间展开

神经网络怎么做到“看图作文”？



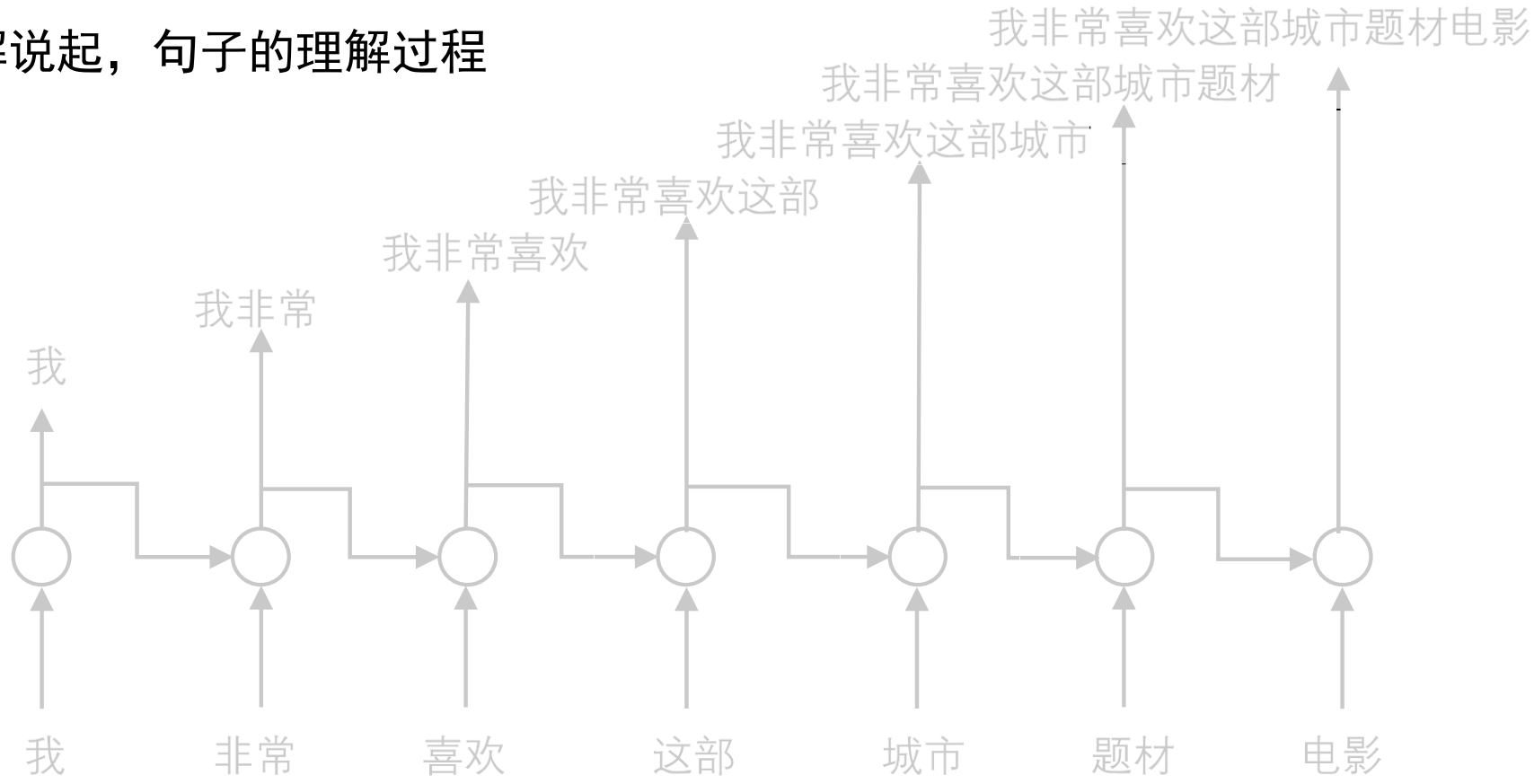




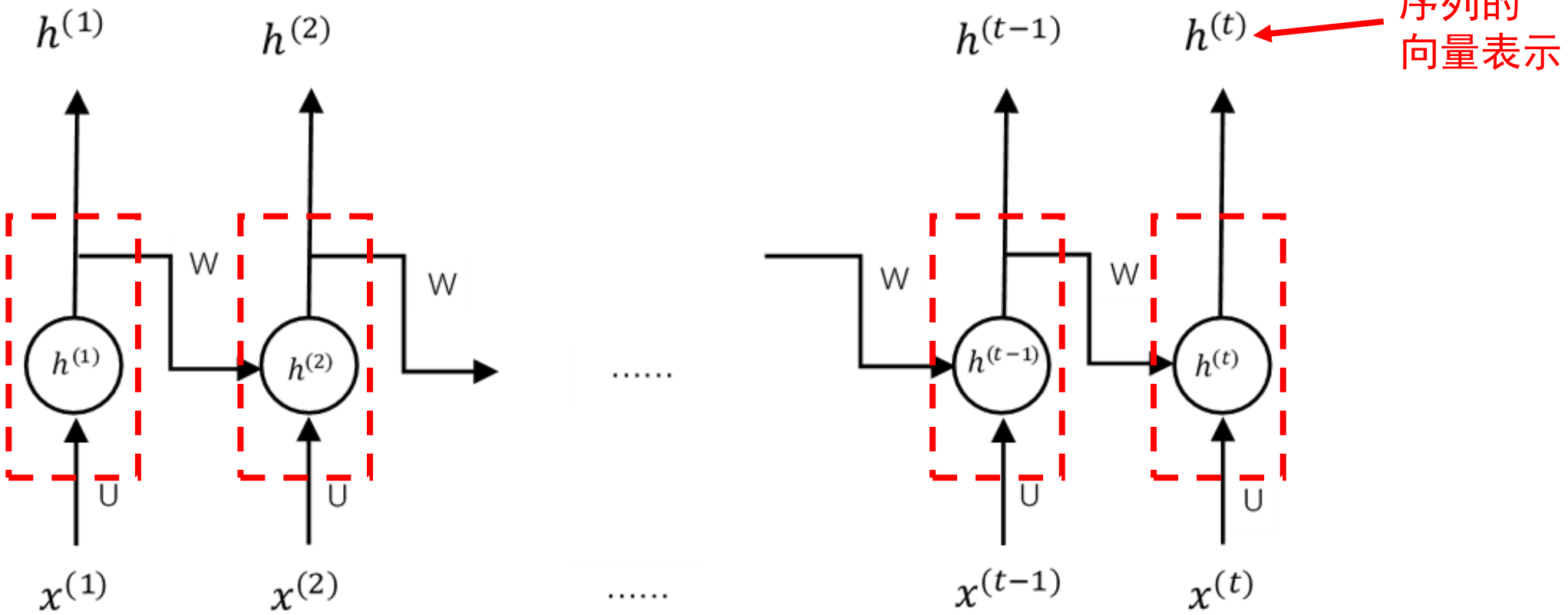
CNN_{θ_c}

循环神经网络（RNN）

- Recurrent Neural Network
- 从句子理解说起，句子的理解过程

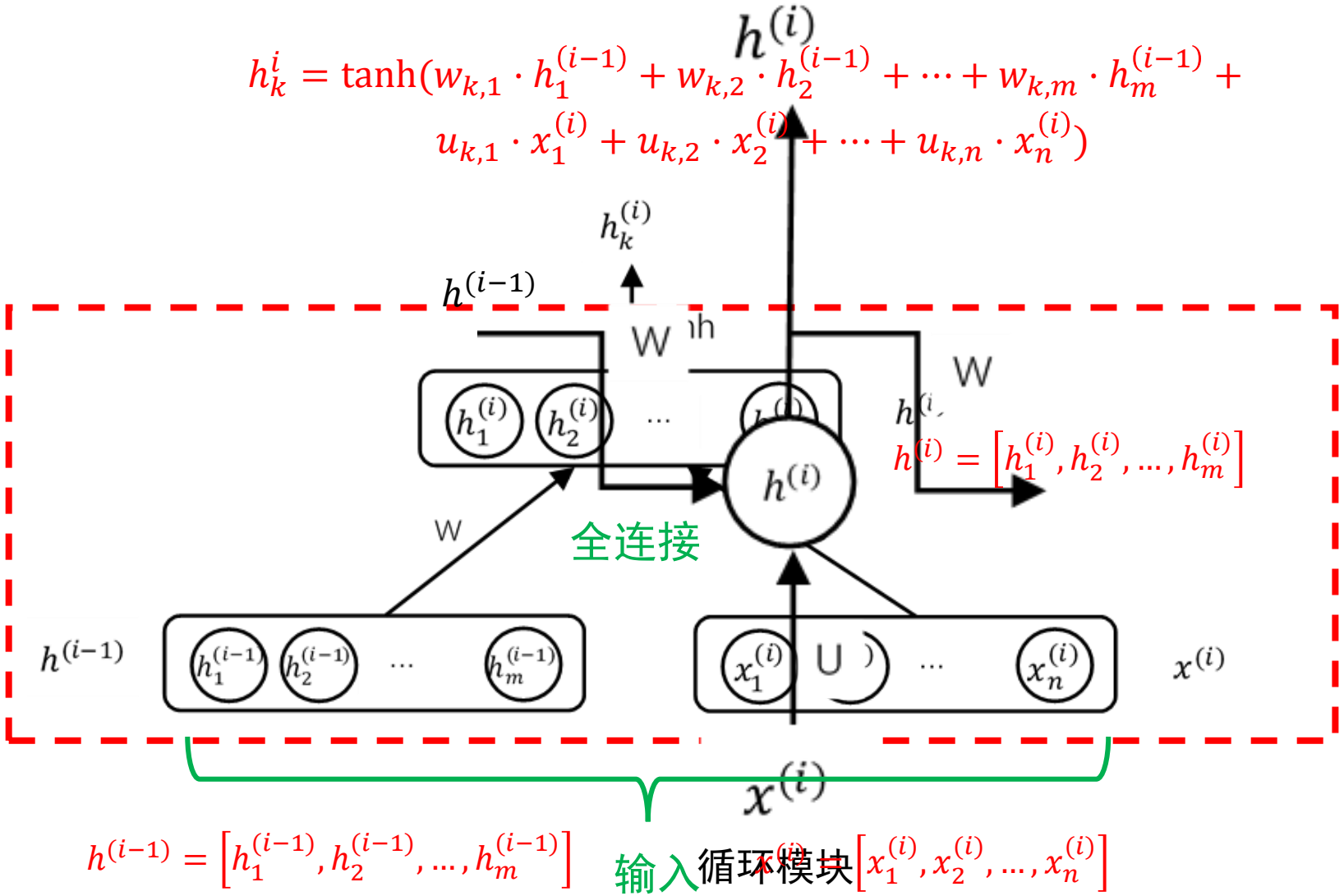


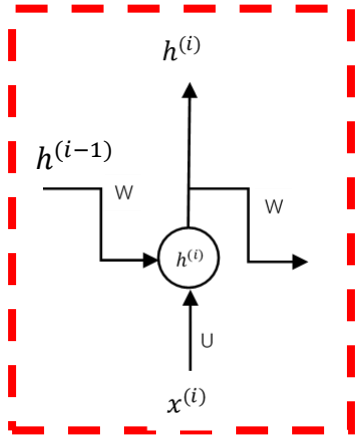
循环神经网络学习序列的编码表示



循环神经网络中的循环模块

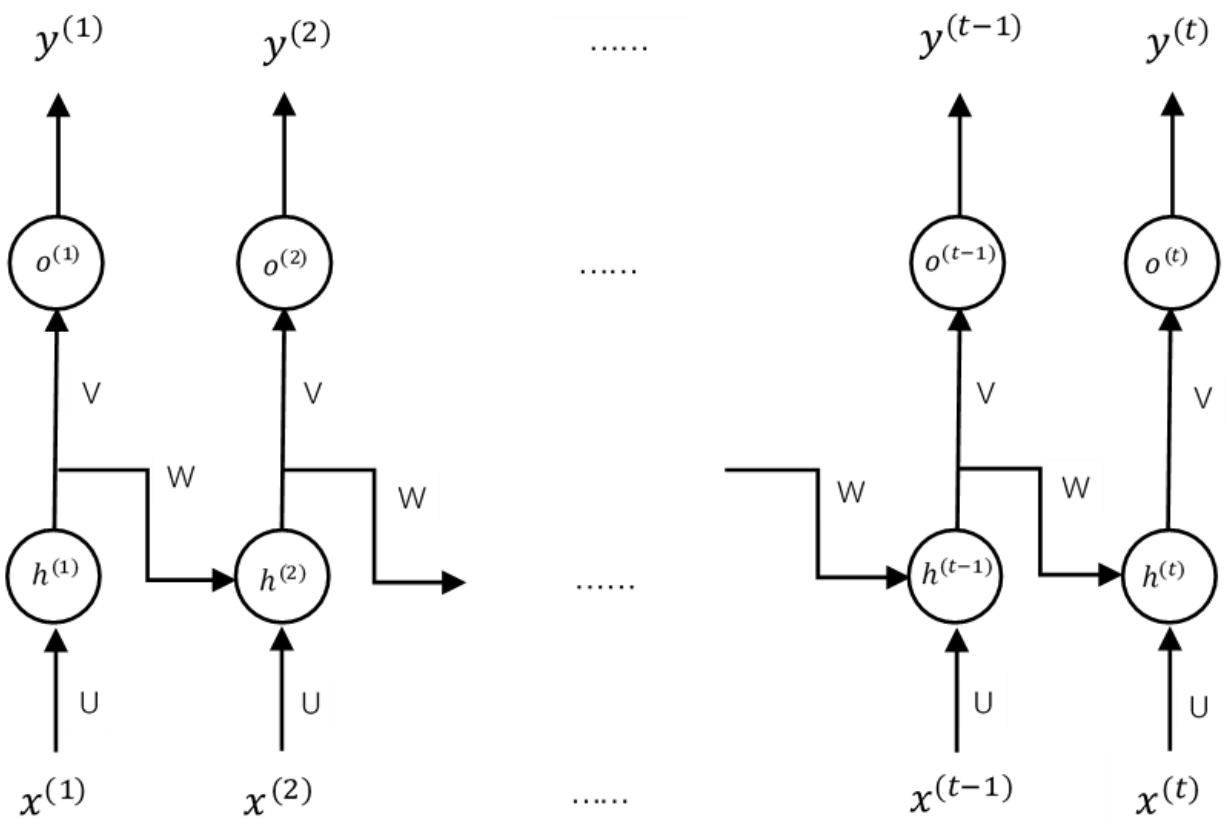
$$h_k^i = \tanh(w_{k,1} \cdot h_1^{(i-1)} + w_{k,2} \cdot h_2^{(i-1)} + \cdots + w_{k,m} \cdot h_m^{(i-1)} + u_{k,1} \cdot x_1^{(i)} + u_{k,2} \cdot x_2^{(i)} + \cdots + u_{k,n} \cdot x_n^{(i)})$$





循环模块

循环神经网络的一般结构（含输出）

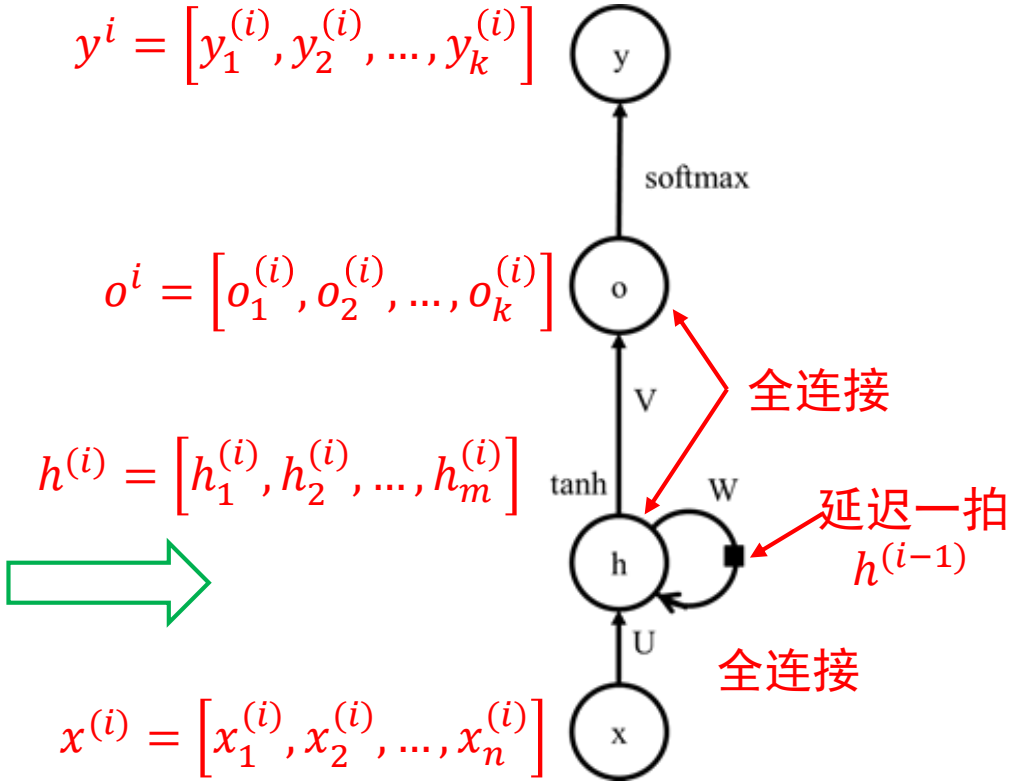


$$y^i = \left[y_1^{(i)}, y_2^{(i)}, \dots, y_k^{(i)}\right]$$

$$o^i = \left[o_1^{(i)}, o_2^{(i)}, \dots, o_k^{(i)}\right]$$

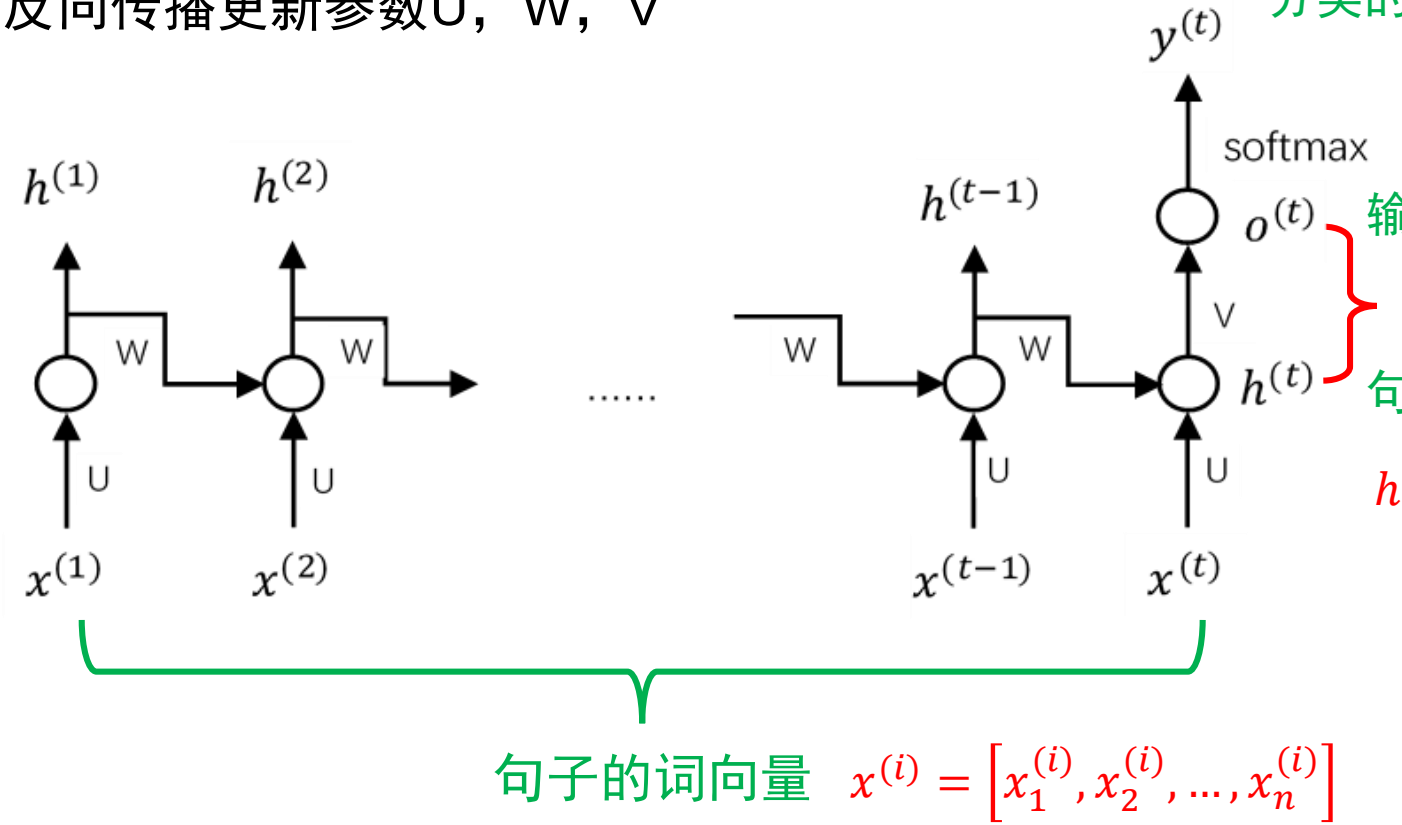
$$h^{(i)} = \left[h_1^{(i)}, h_2^{(i)}, \dots, h_m^{(i)}\right]$$

$$x^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_n^{(i)}\right]$$



循环神经网络的训练

- 与具体任务结合
  - 例：情感分类问题
    - 给定一句话（输入），分类为正/负评价（输出标签）
    - 反向传播更新参数U，W，V



分类的概率 $y^t = \left[y_1^{(t)}, y_2^{(t)}\right]$

输出层 $o^t = \left[o_1^{(t)}, o_2^{(t)}\right]$

全连接

句子向量表示

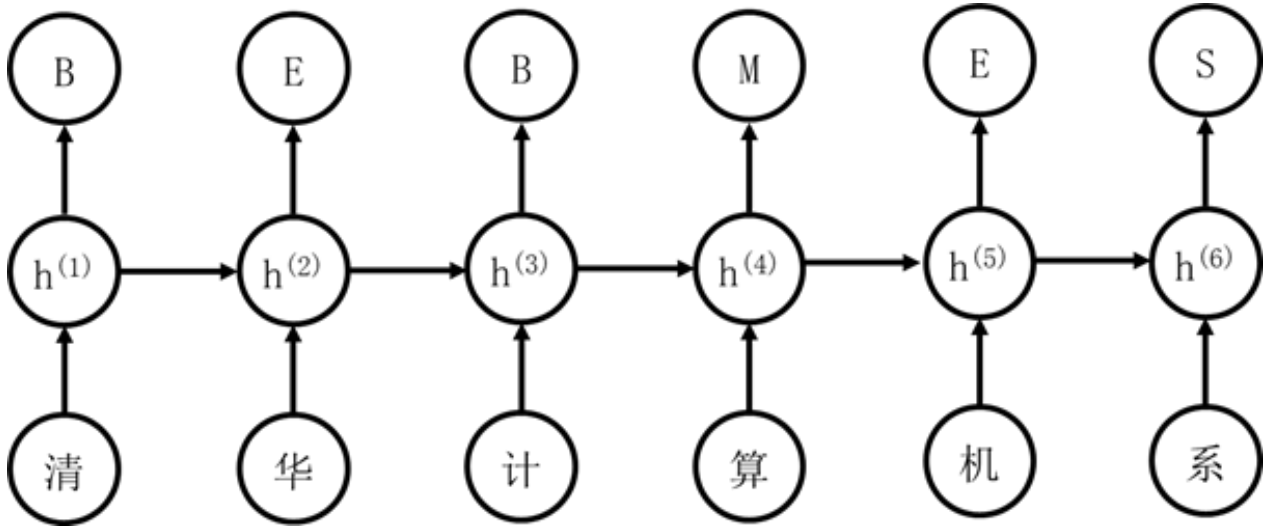
$h^{(t)} = \left[h_1^{(t)}, h_2^{(t)}, \ldots, h_m^{(t)}\right]$

句子的词向量 $x^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \ldots, x_n^{(i)}\right]$

举例：中文分词问题

- 分词问题
  - 清华　大学　计算机　系

- 标记说明
  - B：词首字
  - E：词尾字
  - M：词中字
  - S：单字词



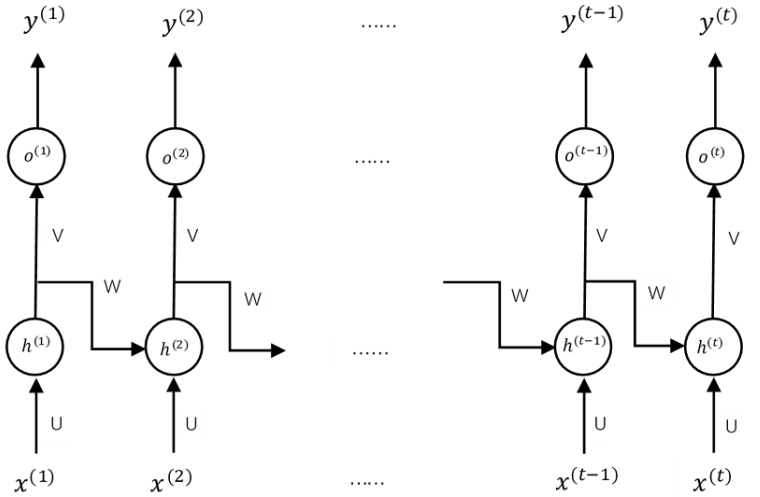
清 华 计 算 机 系

B E　B M E S

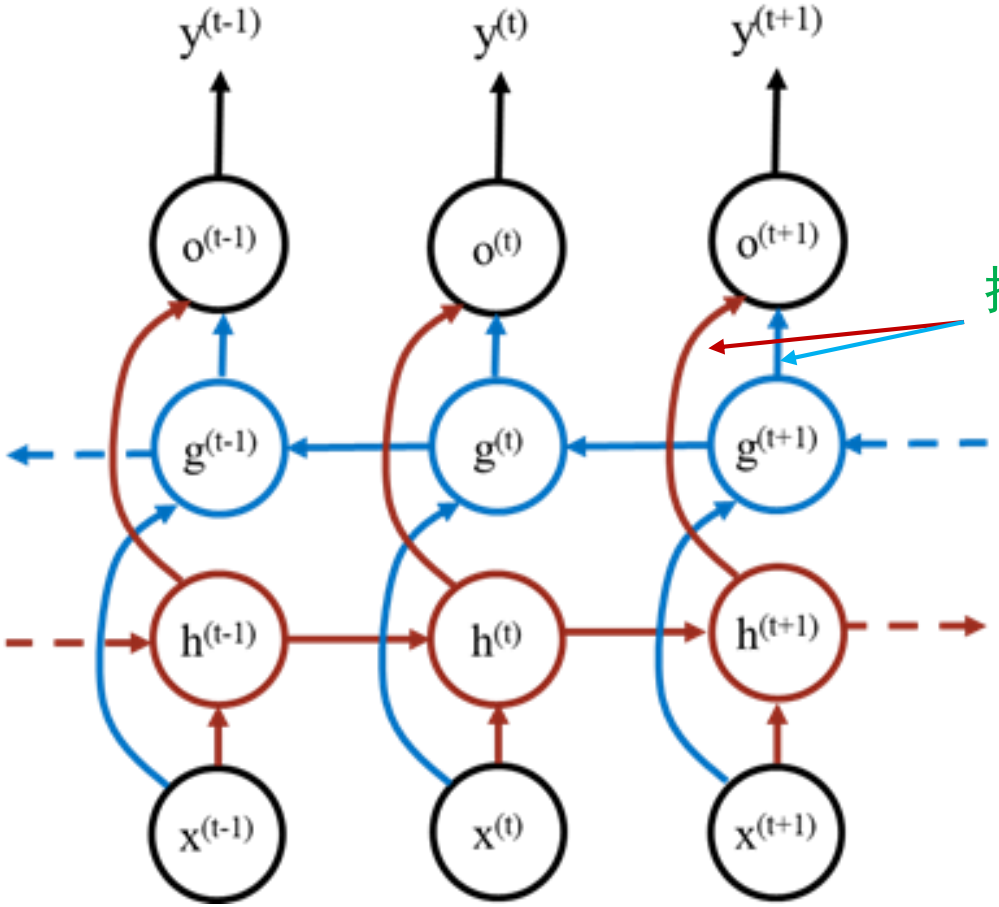
RNN的演进：双向循环神经网络

北京郵電大學 Beijing University of Posts and Telecommunications

- 问题的提出
  - 序列前面的内容被后面的内容淹没



正向处理

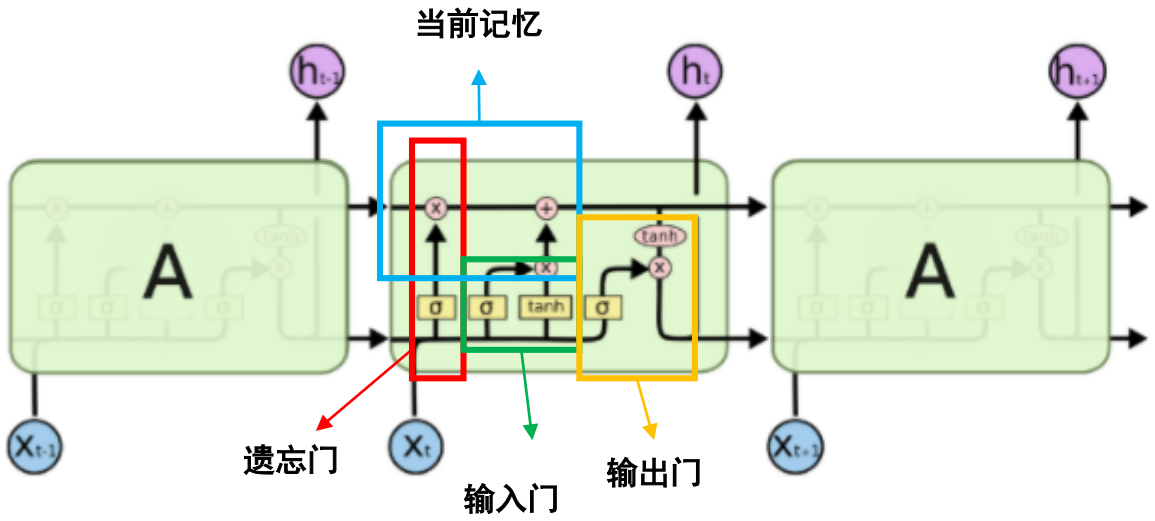


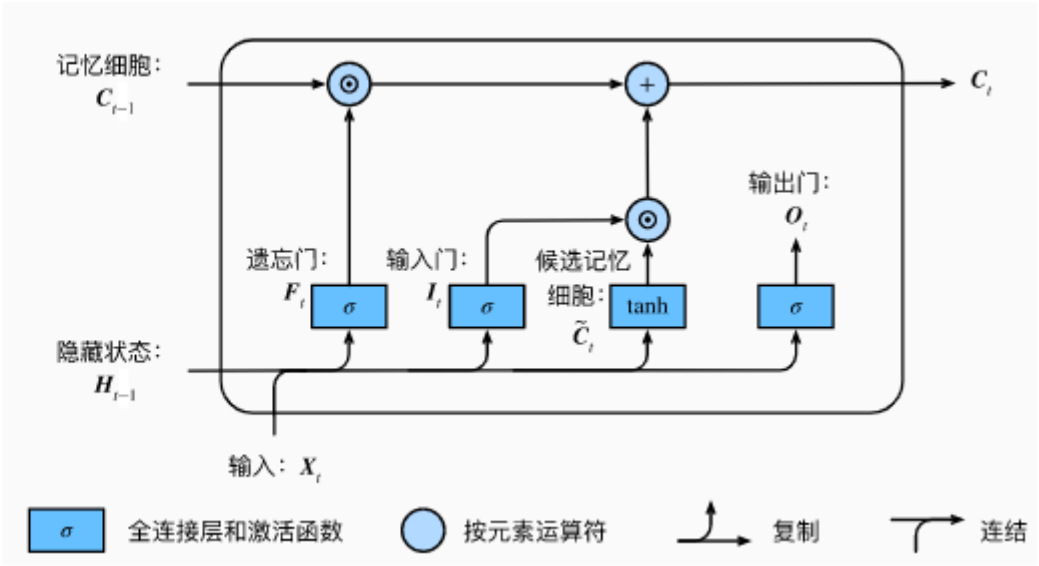
拼接在一起

反向处理

RNN的演进：长短期记忆神经网络

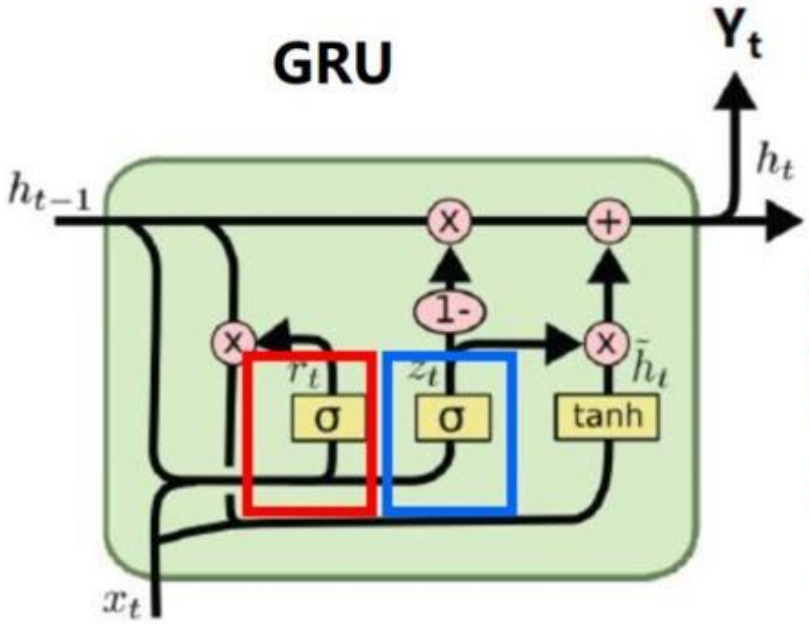
- Long short-term memory
  - RNN的梯度消失与爆炸：基础版本的RNN，我们可以看到，每一时刻的隐藏状态都不仅由该时刻的输入决定，还取决于上一时刻的隐藏层的值，如果一个句子很长，到句子末尾时，它将记不住这个句子的开头的内容详细内容
  - LSTM通过它的“门控装置”有效的缓解了这个问题，这也就是为什么我们现在都在使用LSTM而非普通RNN





RNN的演进：门控循环单元网络

- Gated recurrent unit（GRU）
  - 结构较为复杂，这也引起了研究人员的思考：LSTM结构中到底那一部分是必须的？还可以设计哪些结构允许网络动态的控制时间尺度和不同单元的遗忘行为？对此Kyunghyun Cho et al.等人设计GRU单元(Gated recurrent unit)



更新门： $z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$

重置门： $r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$

新信息： $\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$

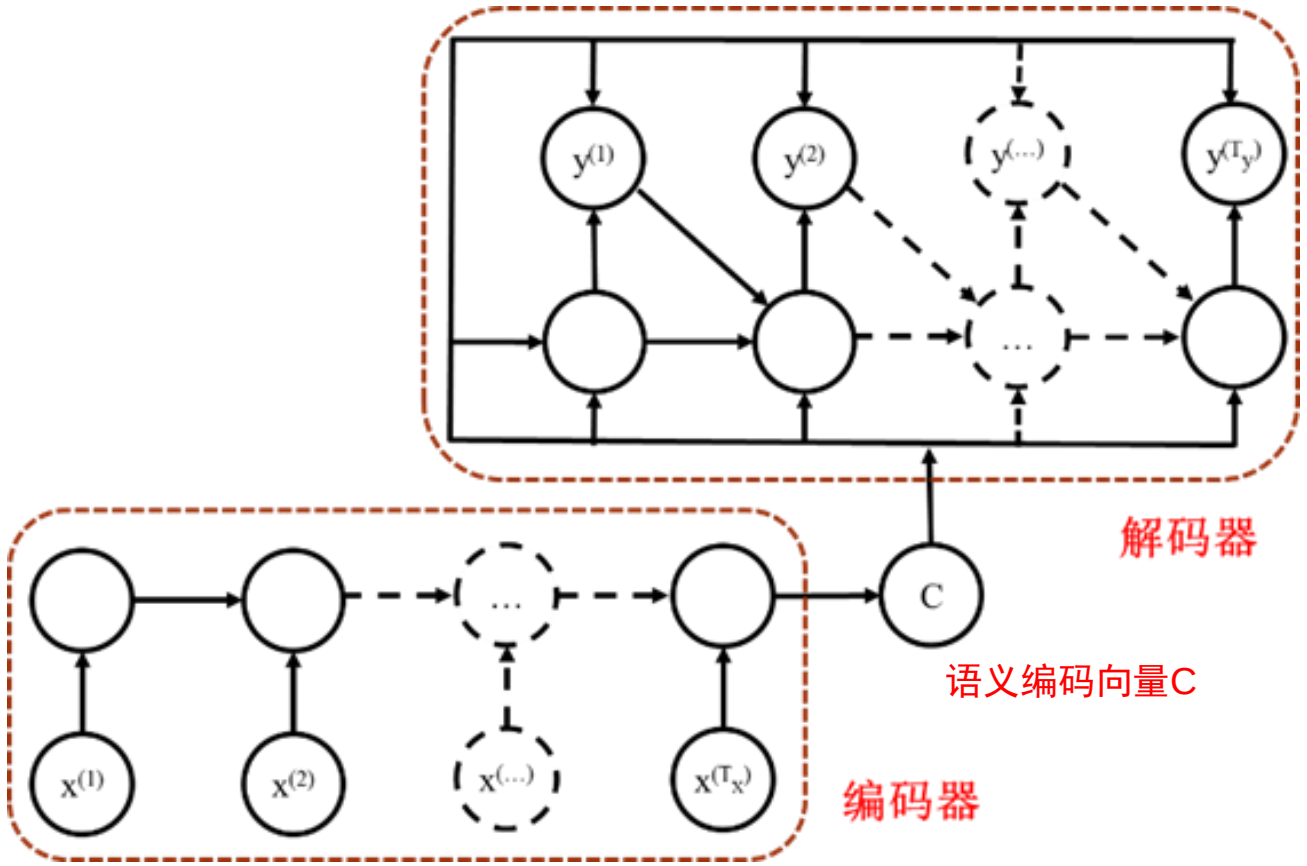
隐状态： $h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$

参数：

W_z ,W_r ,W

编码器-解码器结构

- Encoder-Decoder用途：seq2seq任务
  - 机器翻译
    - 文字到文字
    - 语音到语音
  - 问答系统
    - 问句到答案



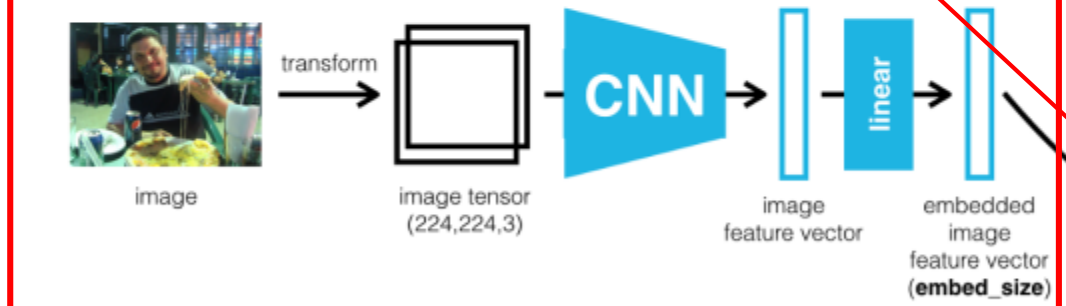
举例：看图说话（image caption）

输出：$y^{(t)} = \left[y_1^{(t)}, y_2^{(t)}, \ldots, y_K^{(t)}\right]$（K为字表长度）

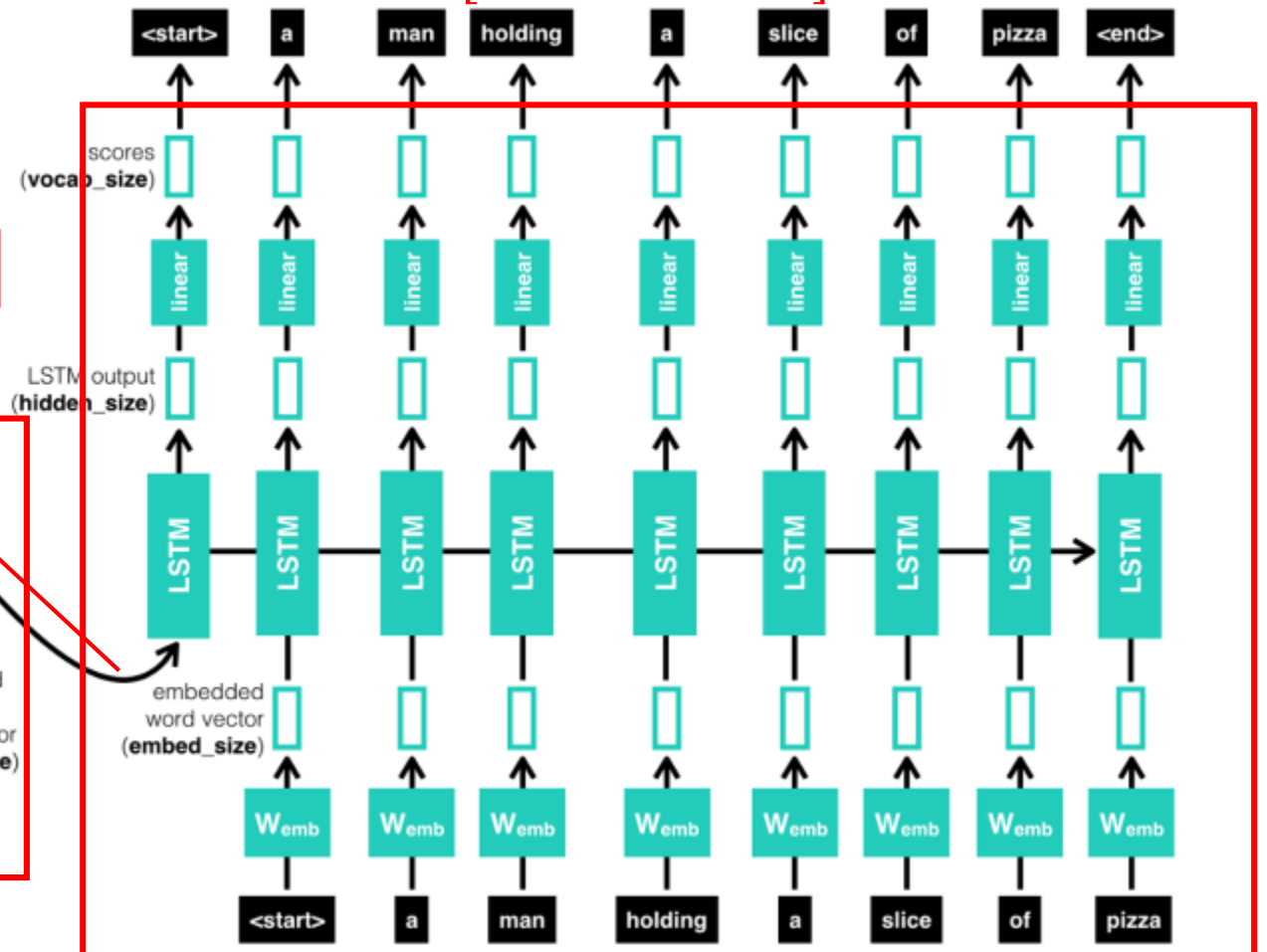
图像编码向量：

$x^{(t)} = \left[x_1^{(t)}, x_2^{(t)}, \ldots, x_n^{(t)}\right]$

输入：二维图像



CNN作为图像编码器，输出为图像的特征



RNN/LSTM作为文字预测的解码器，输出为文字描述

人脑如何处理信息过载？

北京郵電大學 Beijing University of Posts and Telecommunications

神经网络存在的问题

- 计算能力的限制：要记住很多“信息”，模型就变得更复杂
- 优化算法的限制：当输入序列非常长时，模型难以学到合理的表示，信息“记忆”能力并不高

用户_SXn6r0

这是长征几号

时间注意力

ChatGLM

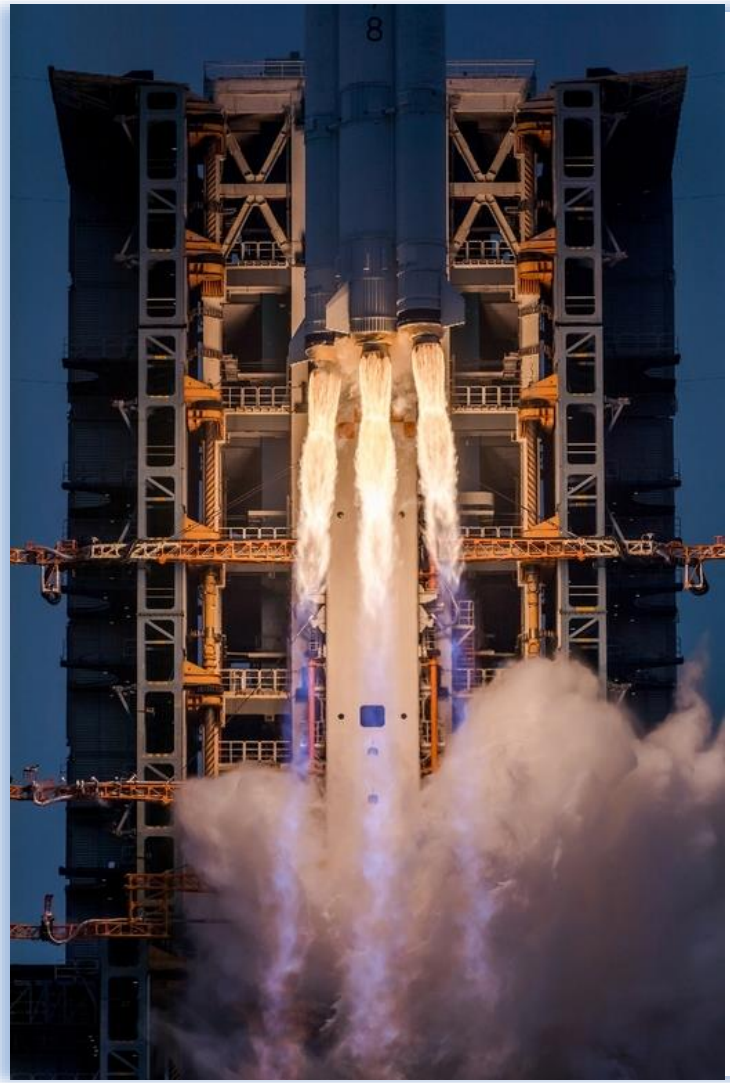
图片中的火箭为长征八号运载火箭（Long March-8），是中国航天科技集团有限公司所属中国运载火箭技术研究院（简称一院）研制的新一代中型运载火箭，主要用于中、低轨卫星发射，同时兼顾高轨卫星发射能力。

长征八号运载火箭于2019年12月27日首飞成功，将试验七号卫星送入预定轨道。该火箭具有零级固体发动机、一级液氢液氧发动机、二级液氧煤油发动机，整流罩直径3.35米，起飞重量约760吨，近地轨道运载能力为14吨，地球同步转移轨道运载能力为25吨。

长征八号运载火箭的研制成功，填补了我国运载火箭中高轨大推力空白，进一步完善了我国运载火箭型谱，对提升我国空间进入能力具有重要意义。

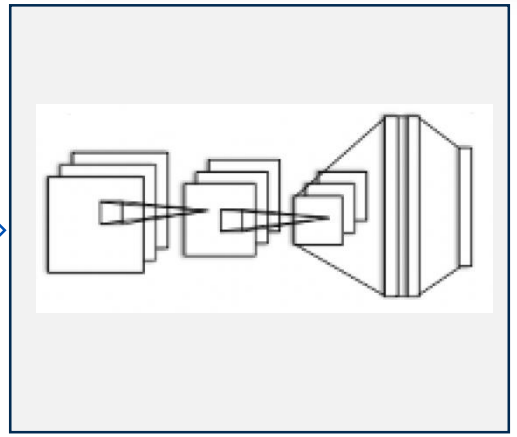
神经网络怎么更好做到“看图作文”？

北京郵電大學 Beijing University of Posts and Telecommunications



无所不能的大模型

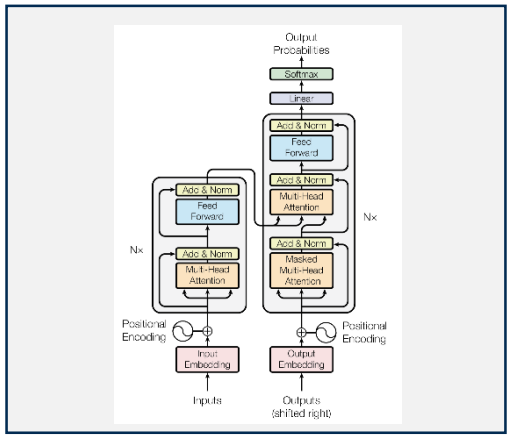
识别图像



卷积神经网络模型

学习图像二维特征

生成文字



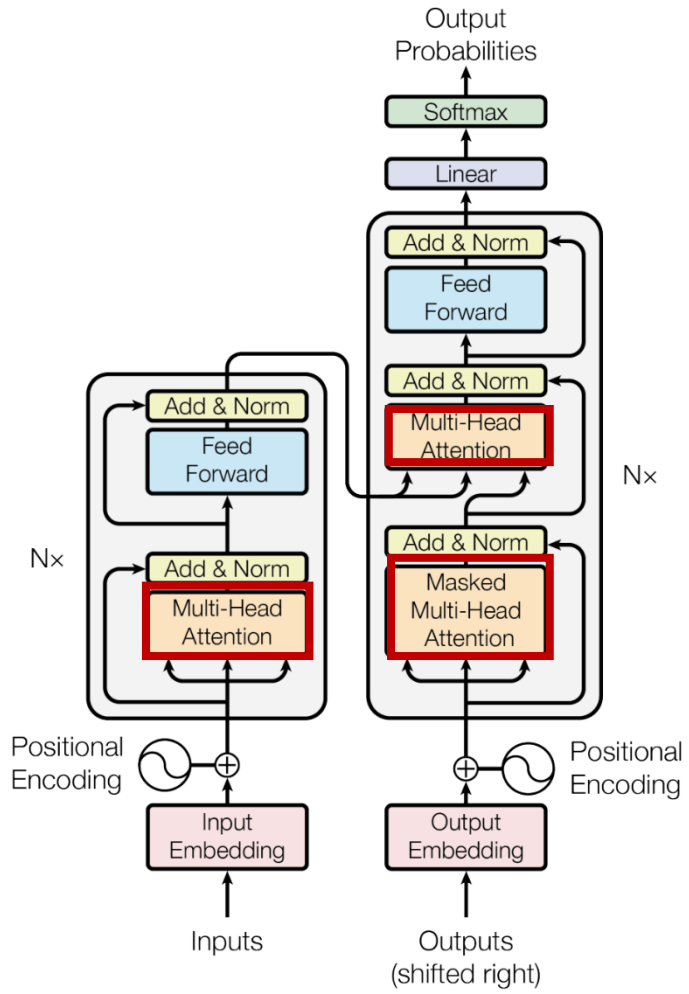
Transformer

学习上下文特征

Transformer

北京郵電大學 Beijing University of Posts and Telecommunications

- 是一种基于自注意力机制的神经网络模型，用于处理序列数据
- 每个时间步的计算只依赖于输入的向量，因此可以实现完全并行的计算



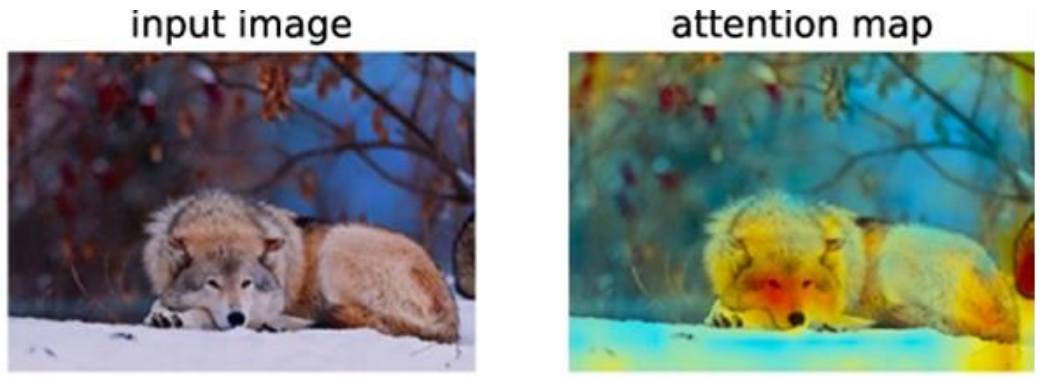
什么是注意力机制？

北京郵電大學 Beijing University of Posts and Telecommunications

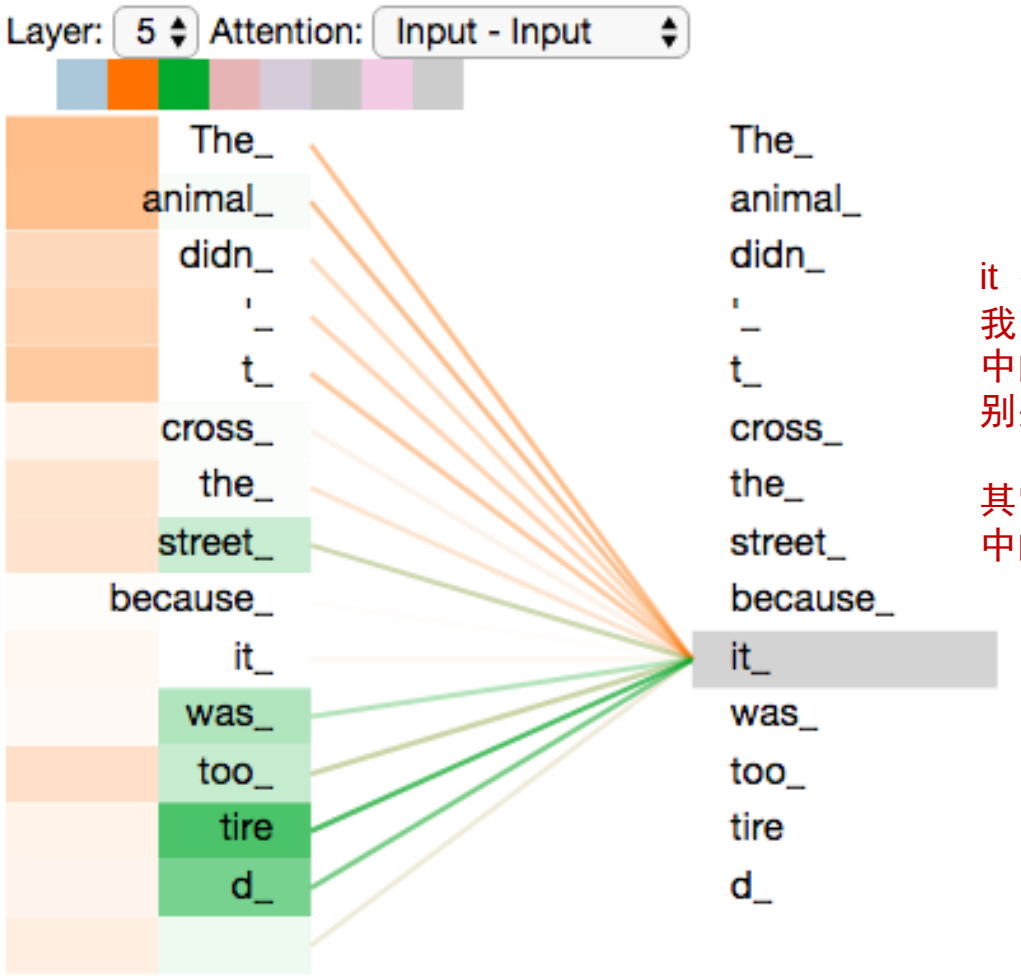
注意力（Attention）机制的本质可以用一句话来总结：上下文决定一切（context is everything）

当人类的视觉机制察觉到一个物体时，通常不会从头到尾地扫视整个场景；一般会根据个人的需求**集中关注特定的部分**。

注意力最早应用在机器视觉领域（CV，Computer Vision），后来才应用到NLP和LLM领域。



我们第一眼应该是看到一只动物，然后，眼睛会先注意到动物的脸，然后得出初步结论，这应该是一只狼；就像右边注意力图所示，颜色更深的部分表示一般是我们人类最先看见（注意）的。



it 指代的是什么呢？我们需要通过上下文中的词语加强对it的特别关注

其它词语会影响it在文中的含义

什么是注意力机制？

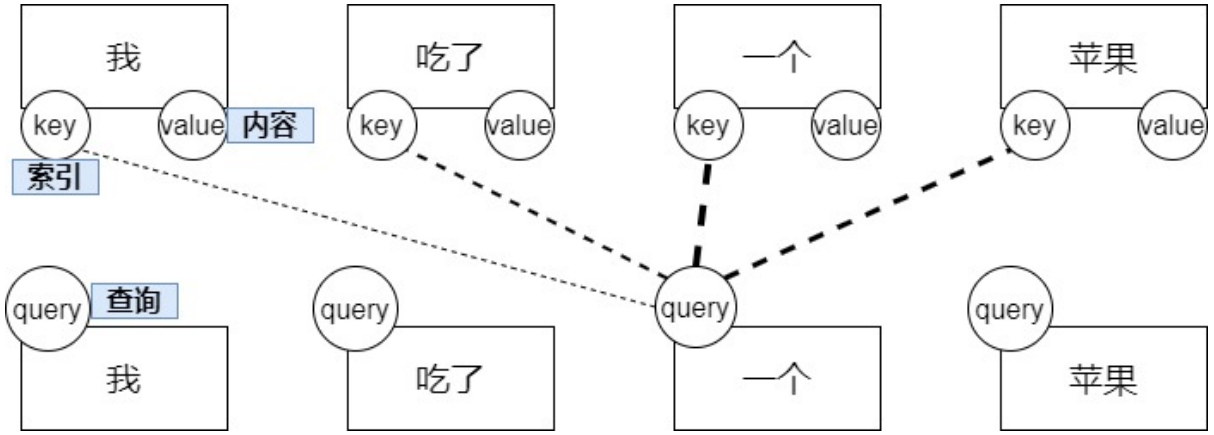
北京郵電大學 Beijing University of Posts and Telecommunications

注意力（Attention）机制由Bengio团队2015年提出。

模仿人类认知注意力的技术，他通过计算输入序列中每个元素的重要性，从而动态地调整模型的关注点，增强神经网络对输入数据中某些部分的关注，同时减弱其他不分的权重，从而将网络的注意力聚焦于数据中最重要的一小部分（通过加权求和进行上下文感知）

Q：查询矩阵，要查找的单词内容；
K：键矩阵，单词的字典索引；
V：值矩阵，每个单词的实际内容；

如果Q=K=V，就是自注意力（Self attention）



Published as a conference paper at ICLR 2015

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dzmitry Bahdanau
Jacobs University Bremen, Germany

KyungHyun Cho **Yoshua Bengio***
Université de Montréal

ABSTRACT

Neural machine translation is a recently proposed approach to machine translation. Unlike the traditional statistical machine translation, the neural machine translation aims at building a single neural network that can be jointly tuned to maximize the translation performance. The models proposed recently for neural machine translation often belong to a family of encoder–decoders and encode a source sentence into a fixed-length vector from which a decoder generates a translation. In this paper, we conjecture that the use of a fixed-length vector is a bottleneck in improving the performance of this basic encoder–decoder architecture, and propose to extend this by allowing a model to automatically (soft-)search for parts of a source sentence that are relevant to predicting a target word, without having to form these parts as a hard segment explicitly. With this new approach, we achieve a translation performance comparable to the existing state-of-the-art phrase-based system on the task of English-to-French translation. Furthermore, qualitative analysis reveals that the (soft-)alignments found by the model agree well with our intuition.

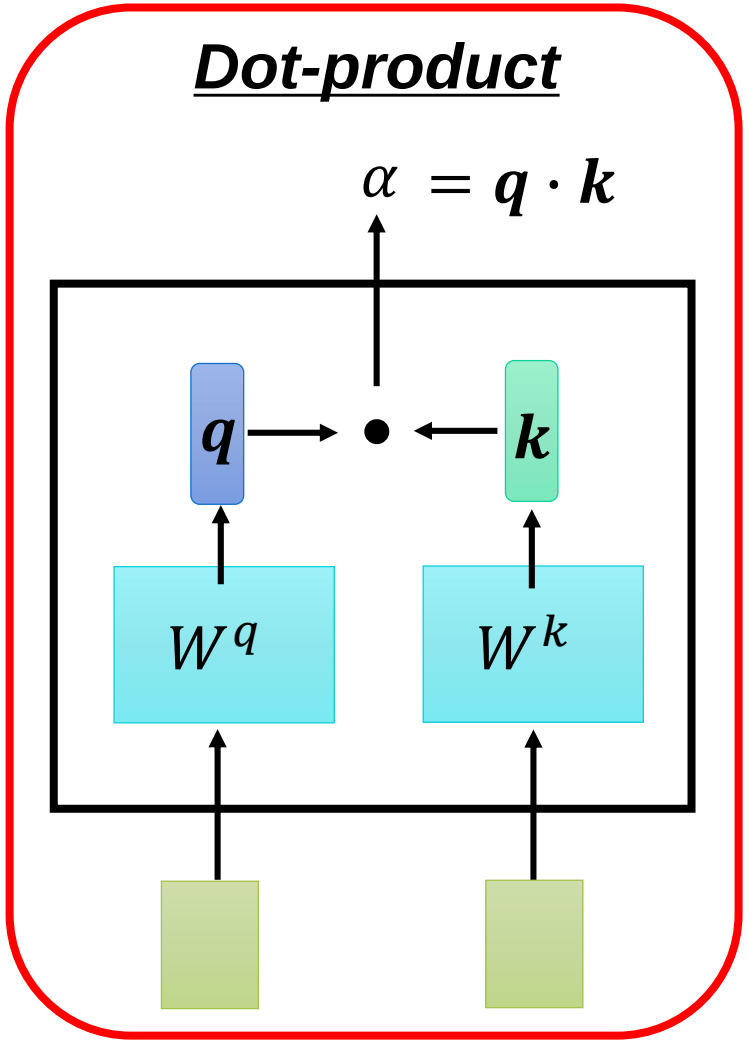
1 INTRODUCTION

Neural machine translation is a newly emerging approach to machine translation, recently proposed by Kalchbrenner and Blunsom (2013), Sutskever *et al.* (2014) and Cho *et al.* (2014b). Unlike the traditional phrase-based translation system (see, e.g., Koehn *et al.*, 2003) which consists of many small sub-components that are tuned separately, neural machine translation attempts to build and train a single, large neural network that reads a sentence and outputs a correct translation.

Most of the proposed neural machine translation models belong to a family of *encoder–decoders* (Sutskever *et al.*, 2014; Cho *et al.*, 2014a), with an encoder and a decoder for each language, or involve a language-specific encoder applied to each sentence whose outputs are then compared (Hermann and Blunsom, 2014). An encoder neural network reads and encodes a source sentence into a fixed-length vector. A decoder then outputs a translation from the encoded vector. The whole encoder–decoder system, which consists of the encoder and the decoder for a language pair, is jointly trained to maximize the probability of a correct translation given a source sentence.

A potential issue with this encoder–decoder approach is that a neural network needs to be able to compress all the necessary information of a source sentence into a fixed-length vector. This may

注意力机制（数学表达）



$\alpha_{1,2} = q^1 \cdot k^2$ $\alpha_{1,3} = q^1 \cdot k^3$ $\alpha_{1,4} = q^1 \cdot k^4$

attention score

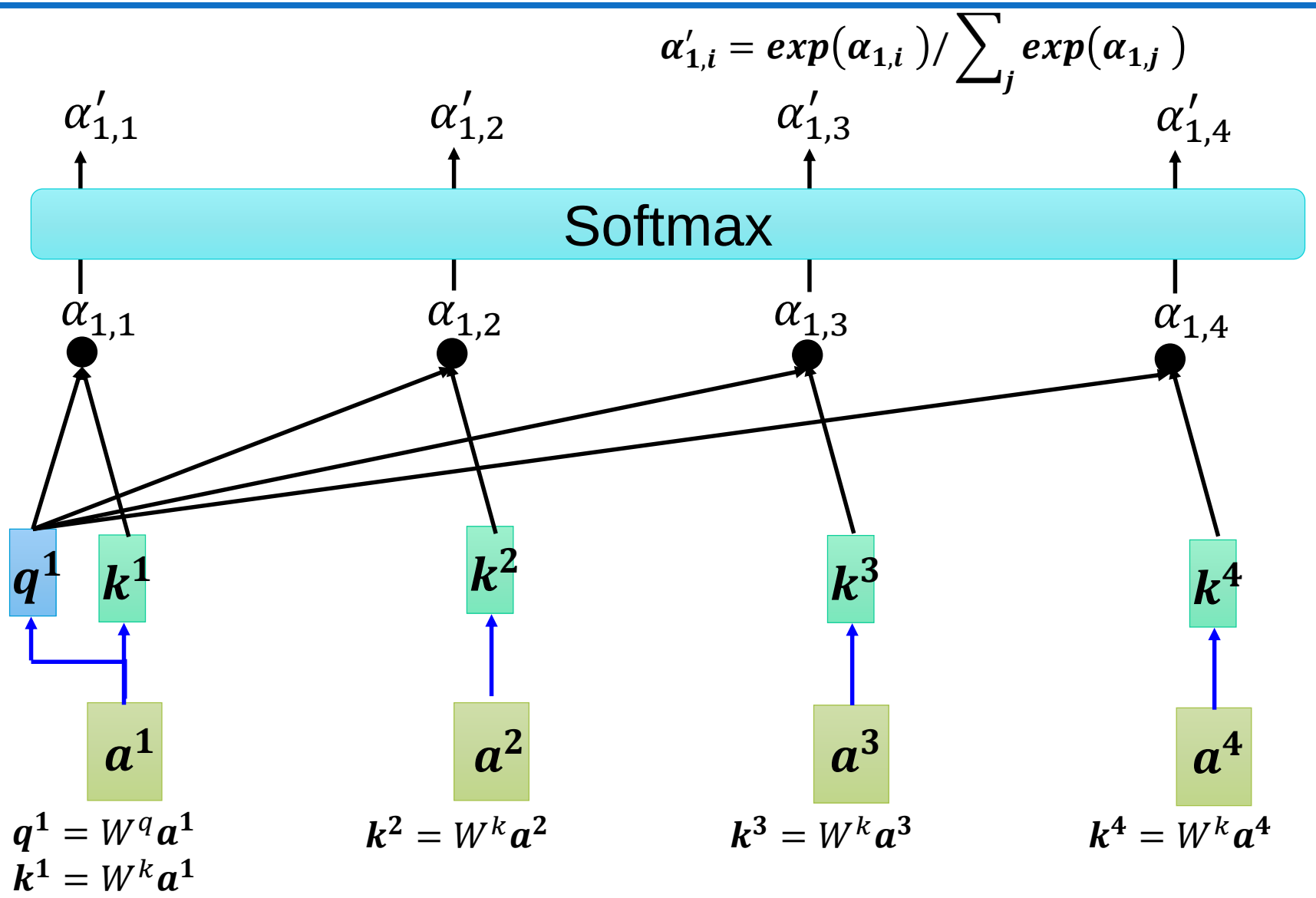
$\alpha_{1,2}$ $\alpha_{1,3}$ $\alpha_{1,4}$

q^1 query k^2 key k^3 k^4

a^1 a^2 a^3 a^4

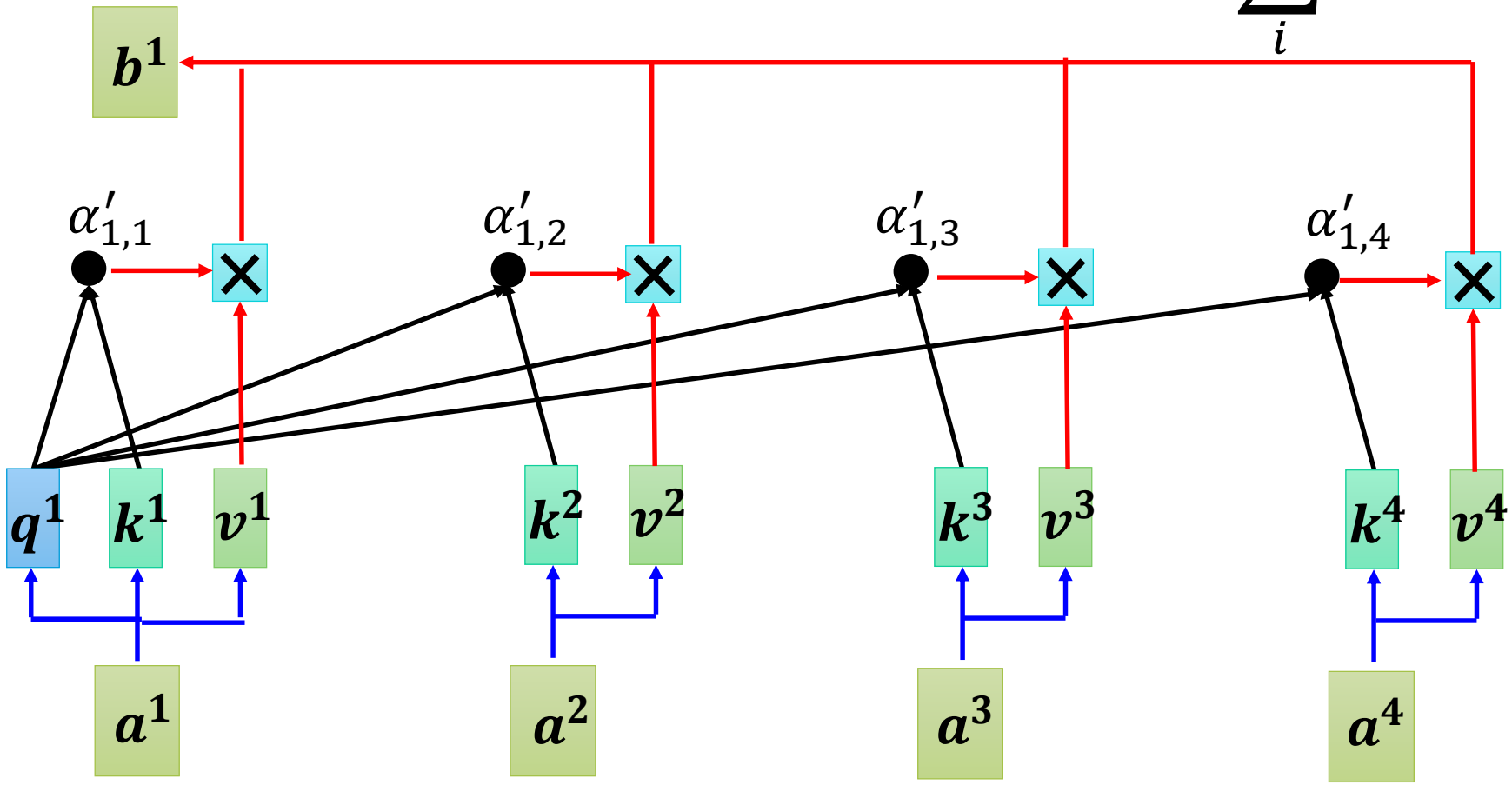
$q^1 = W^q a^1$ $k^2 = W^k a^2$ $k^3 = W^k a^3$ $k^4 = W^k a^4$

注意力机制（数学表达）


$\alpha'_{1,i} = exp(\alpha_{1,i}) / \sum_j exp(\alpha_{1,j})$
$\alpha'_{1,1}$
$\alpha'_{1,2}$
$\alpha'_{1,3}$
$\alpha'_{1,4}$
Softmax
$\alpha_{1,1}$
$\alpha_{1,2}$
$\alpha_{1,3}$
$\alpha_{1,4}$
q^1
k^1
k^2
k^3
k^4
a^1
a^2
a^3
a^4
$q^1 = W^q a^1$
$k^1 = W^k a^1$
$k^2 = W^k a^2$
$k^3 = W^k a^3$
$k^4 = W^k a^4$

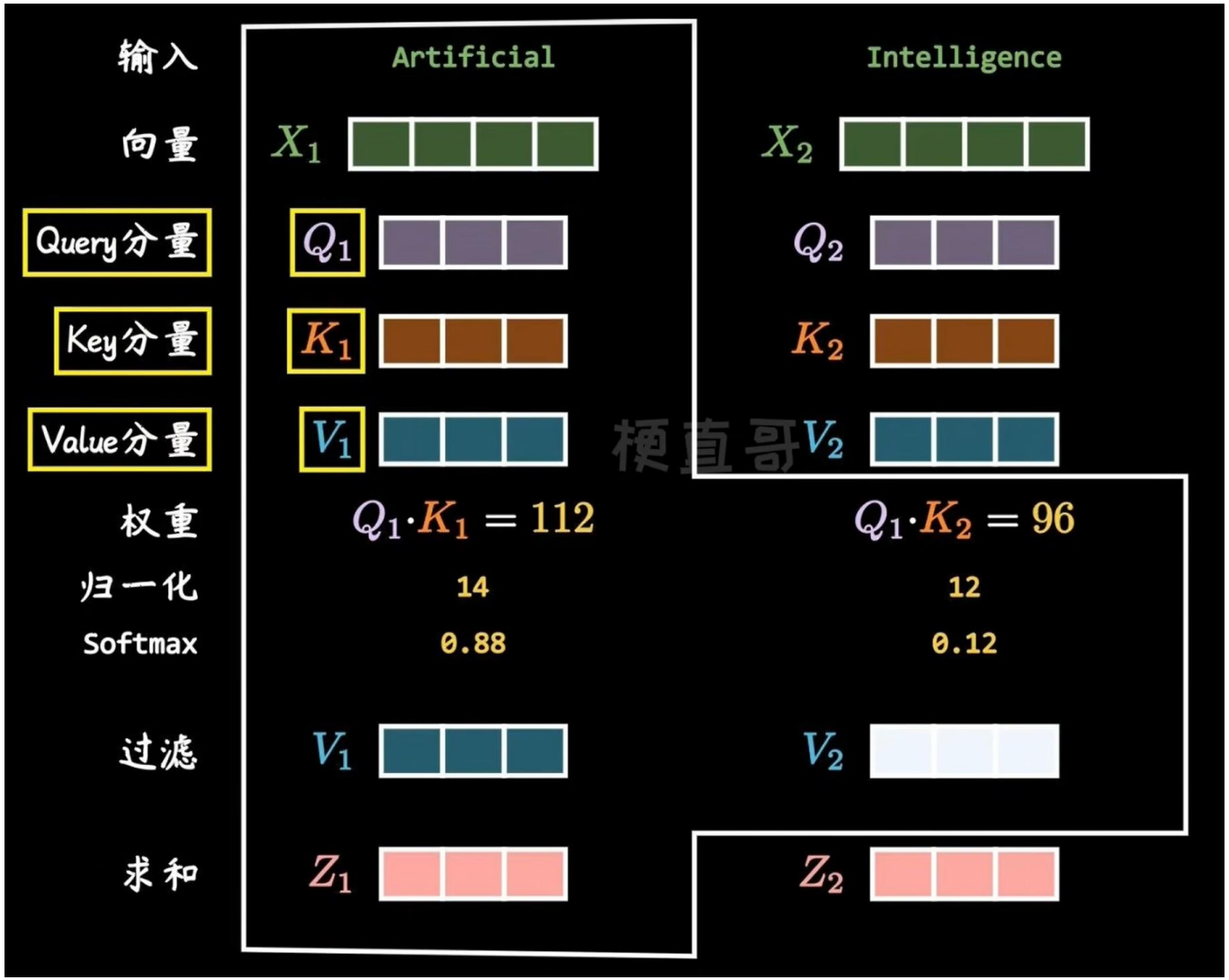
注意力机制（数学表达）

Extract information based on attention scores $b^1 = \sum_i \alpha'_{1,i} v^i$

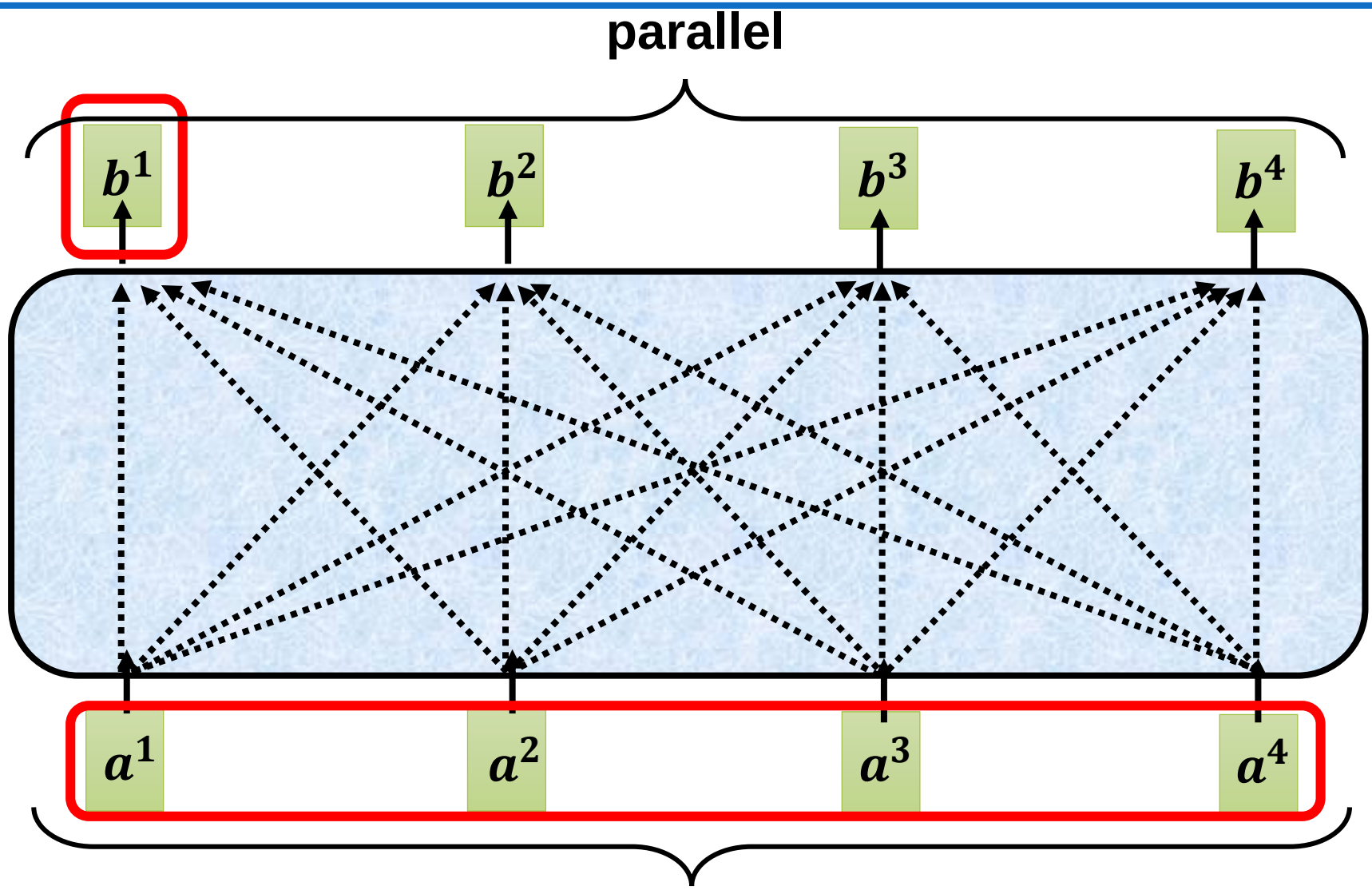


$v^1 = W^v a^1$ $v^2 = W^v a^2$ $v^3 = W^v a^3$ $v^4 = W^v a^4$

注意力机制（数学表达）


输入
Artificial
Intelligence
向量
X_1
X_2
Query分量
Q_1
Q_2
Key分量
K_1
K_2
Value分量
V_1
V_2
梗直哥
权重
$Q_1 \cdot K_1 = 112$
$Q_1 \cdot K_2 = 96$
归一化
14
12
Softmax
0.88
0.12
过滤
V_1
V_2
求和
Z_1
Z_2

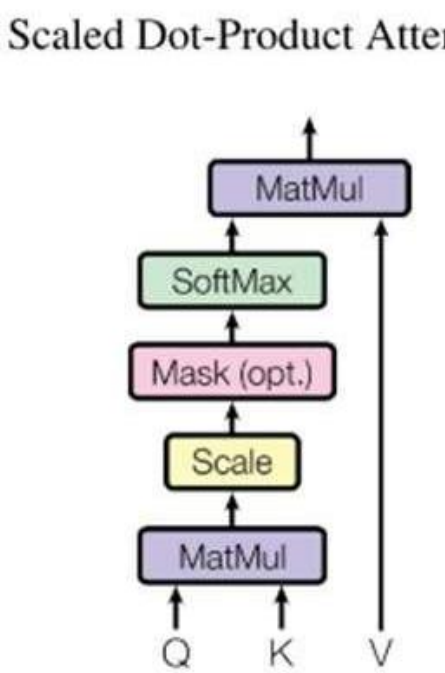
注意力机制


parallel
b^1
b^2
b^3
b^4
a^1
a^2
a^3
a^4
Can be either **input** or **a hidden layer**

注意力机制（数学表达）

$$\text{Attention}(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

如果Q=K=V，就是自注意力（Self attention）



$$\begin{bmatrix} Q \\ K \\ V \end{bmatrix} = \begin{bmatrix} W^q \\ W^k \\ W^v \end{bmatrix} \begin{bmatrix} I \\ I \\ I \end{bmatrix}$$

Parameters to be learned

$$A = K^T \quad Q$$

$$A' \leftarrow A$$

Attention Matrix

$$O = V \quad A'$$

多头自注意力机制

北京郵電大學 Beijing University of Posts and Telecommunications

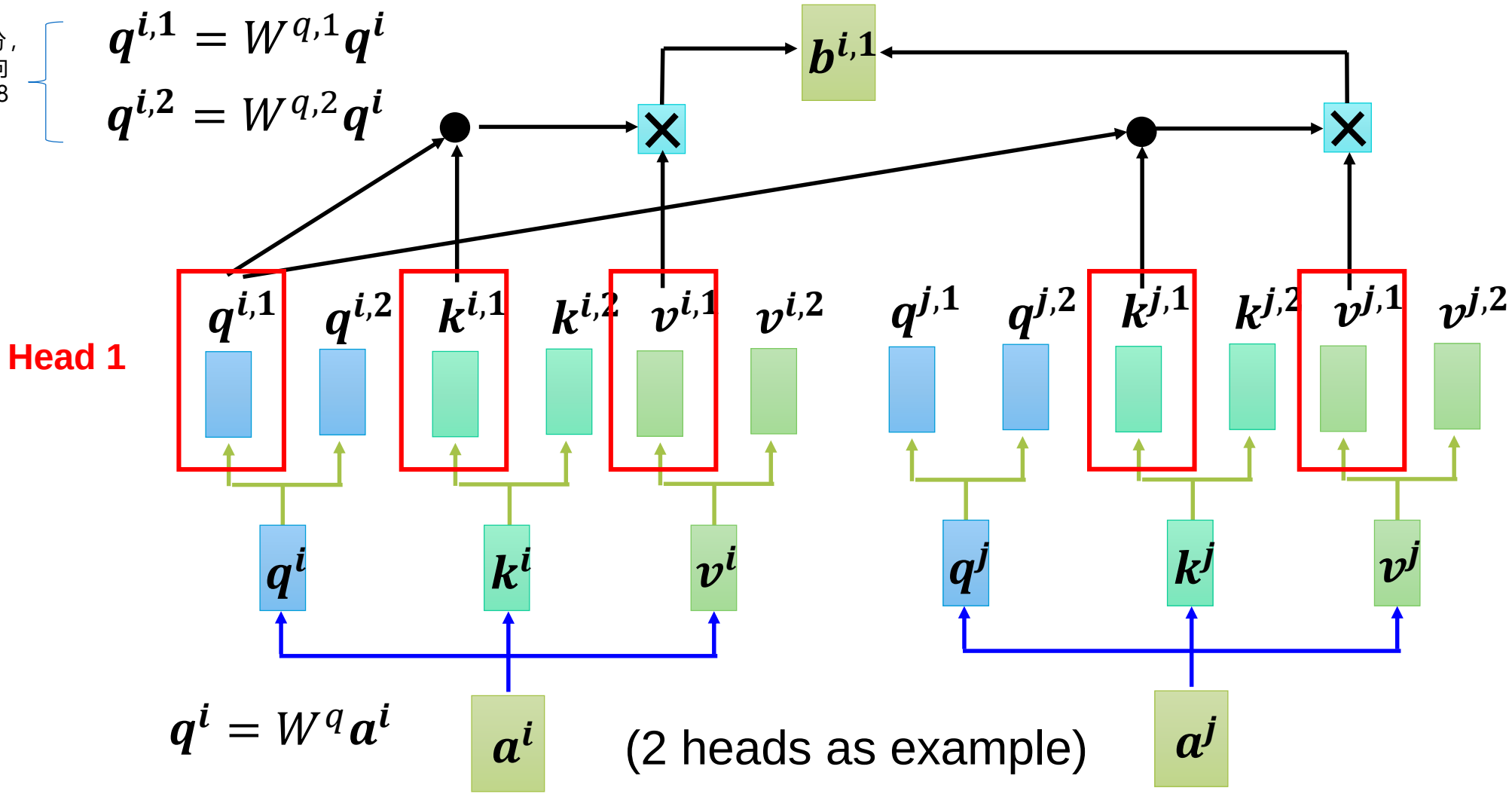
Multi-head Self-attention

Different types of relevance

把q切成2份，比如16维向量分成2个8维的向量

$$q^{i,1} = W^{q,1} q^i$$

$$q^{i,2} = W^{q,2} q^i$$



$$q^i = W^q a^i$$

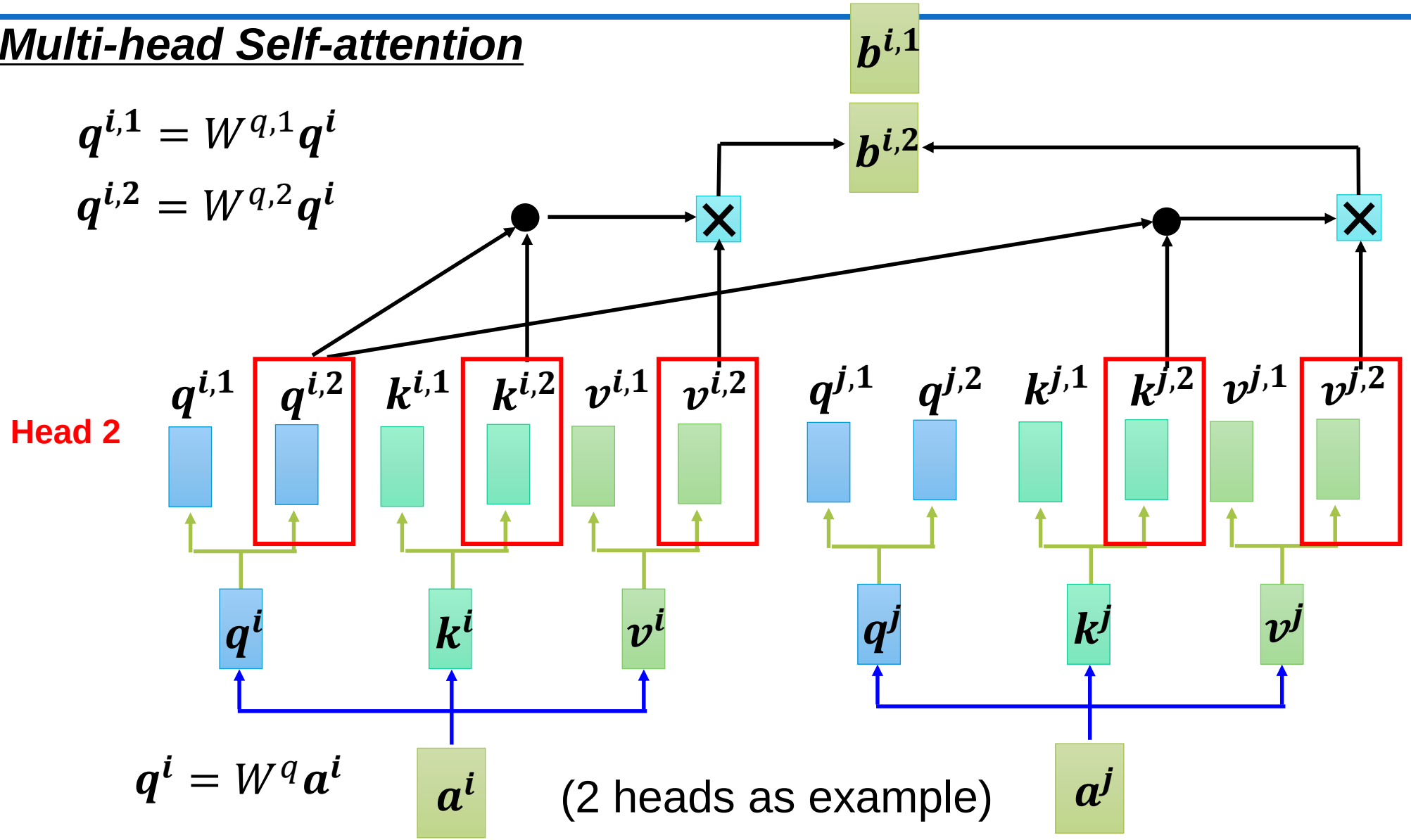
(2 heads as example)

多头自注意力机制 Different types of relevance

Multi-head Self-attention

$$q^{i,1} = W^{q,1} q^i$$

$$q^{i,2} = W^{q,2} q^i$$



$$q^i = W^q a^i$$

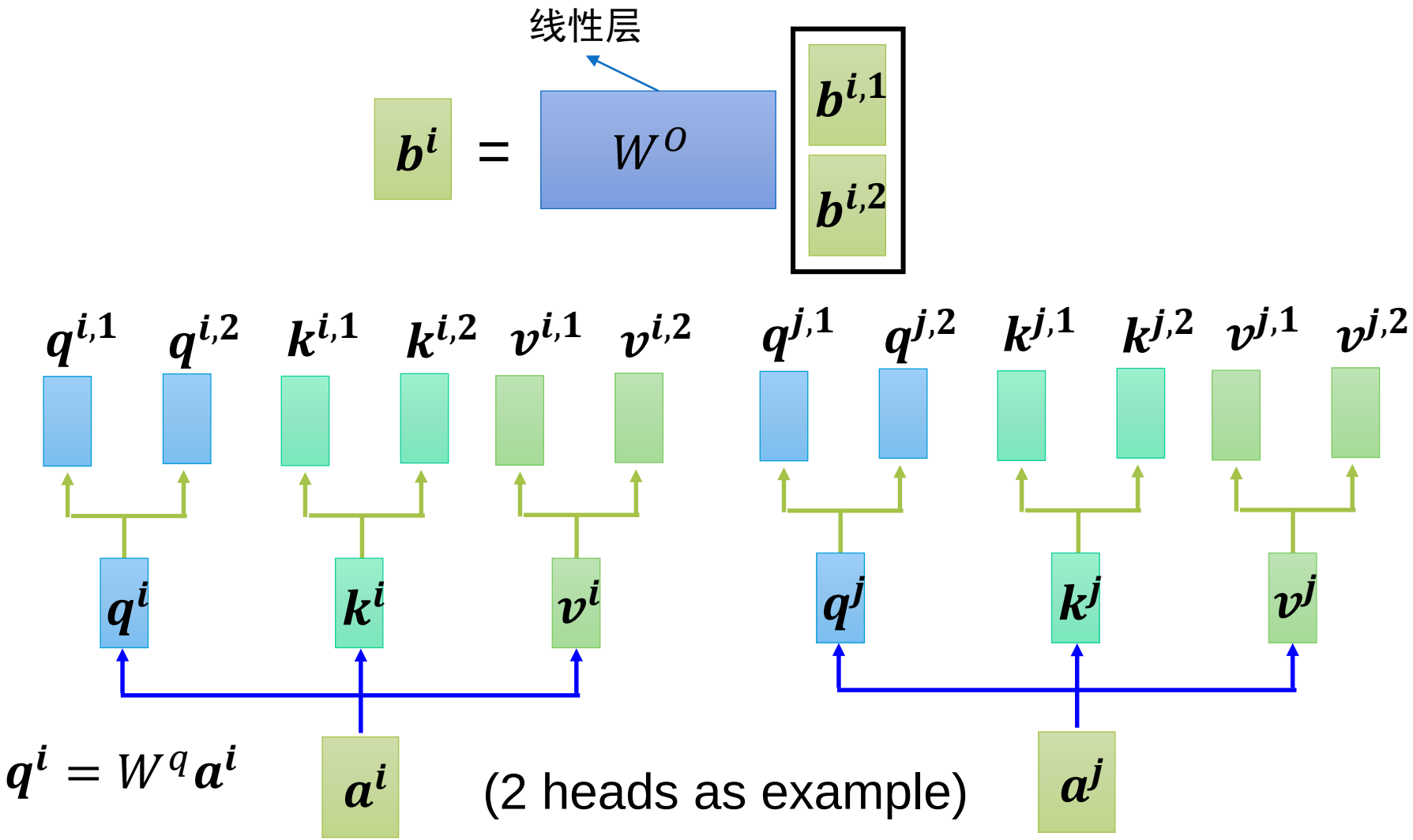
(2 heads as example)

多头自注意力机制

北京郵電大學 Beijing University of Posts and Telecommunications

Multi-head Self-attention

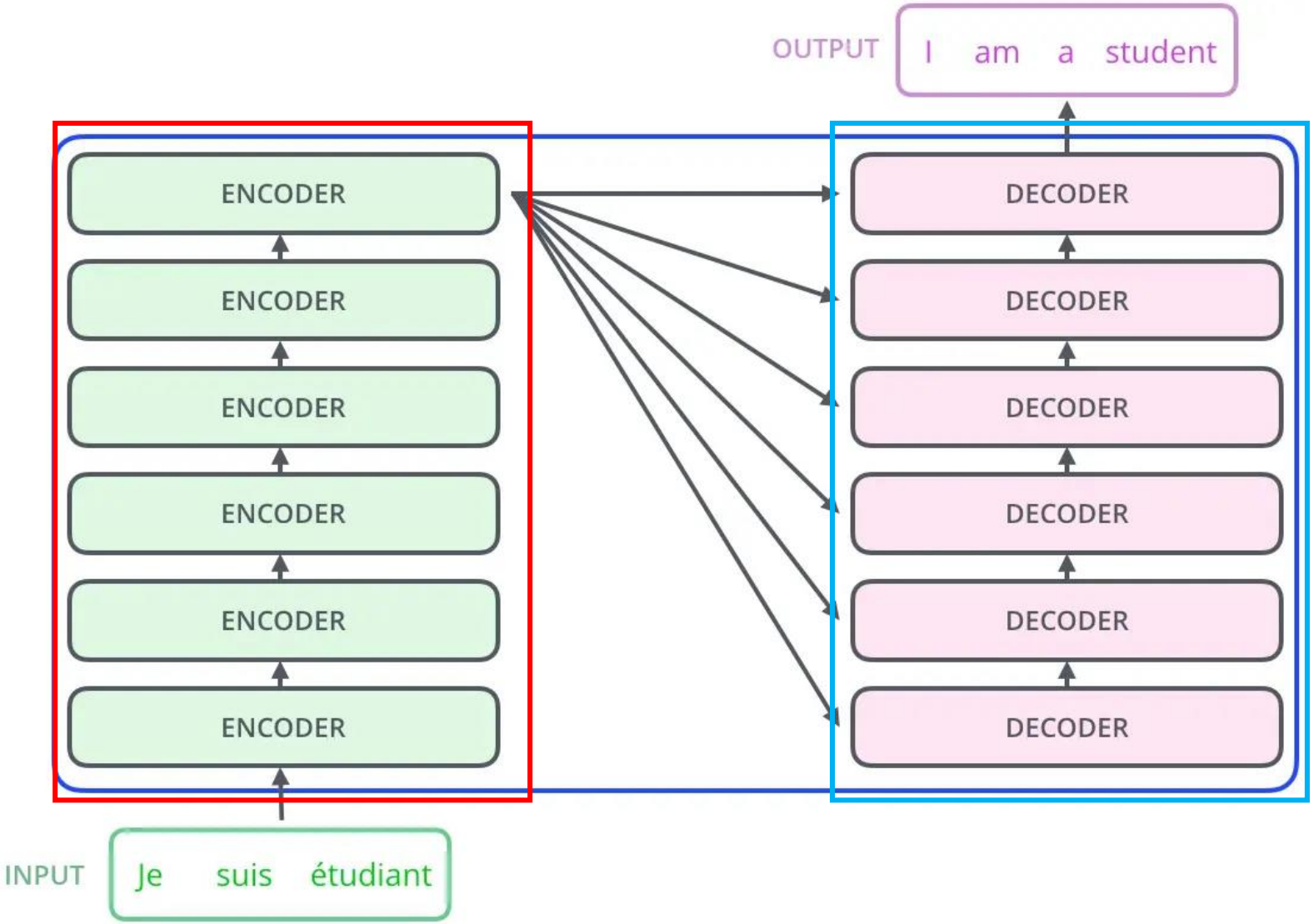
Different types of relevance

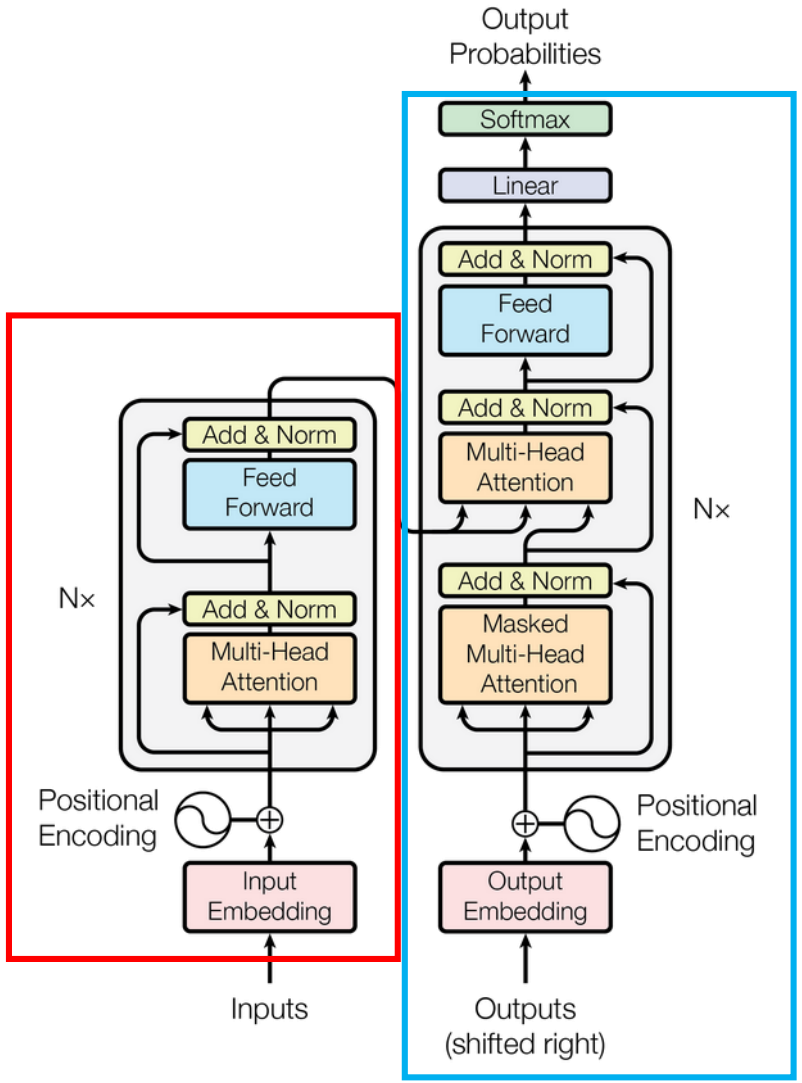


Transformer的基本结构

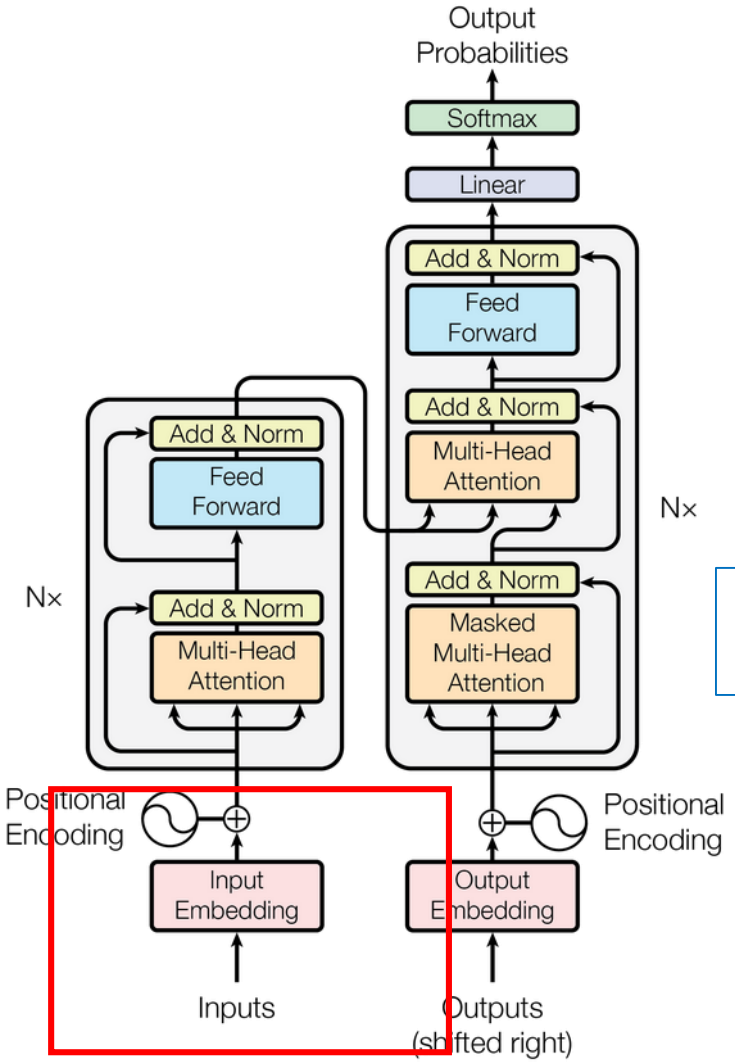
注：代码均为基于Pytorch的实现

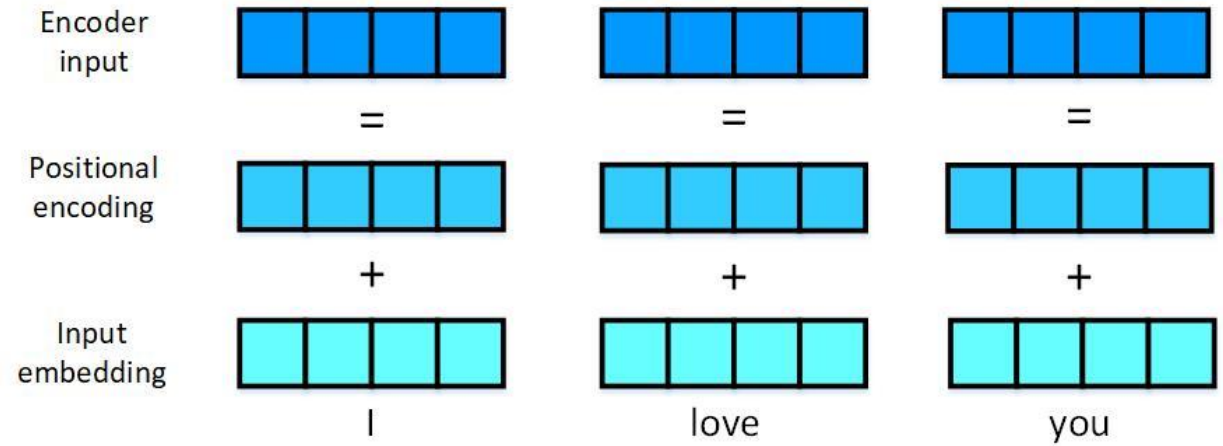
Transformer由Encoder-Decoder组成





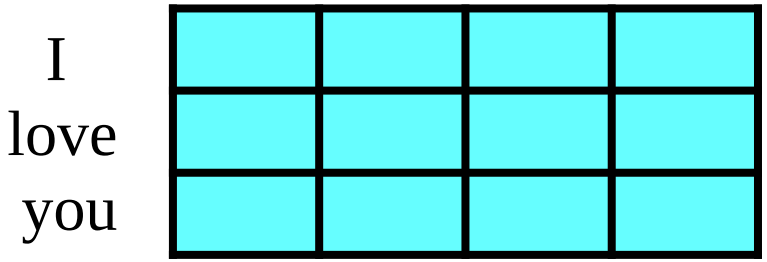
Transformer的输入：位置编码+词向量





$$PE(pos, 2i) = sin(pos/10000^{2i/d_{model}})$$
$$PE(pos, 2i+1) = cos(pos/10000^{2i/d_{model}})$$

pos: 位置
i : 位置编码维度索引



Input X

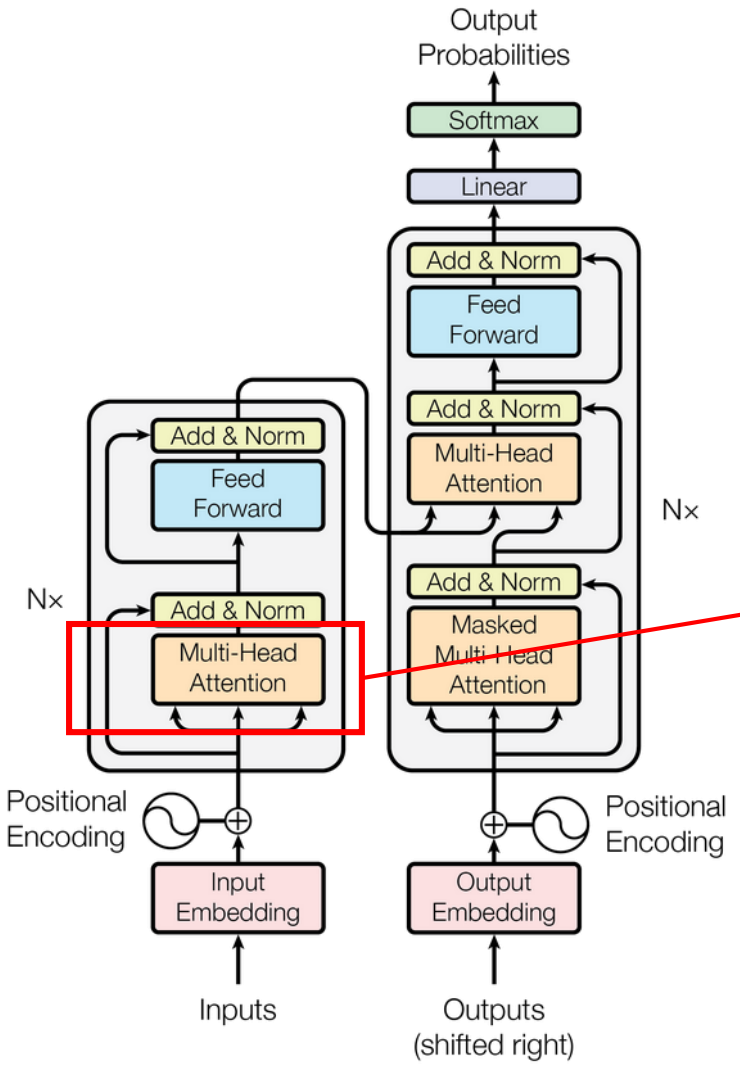
位置编码+词向量源码解读

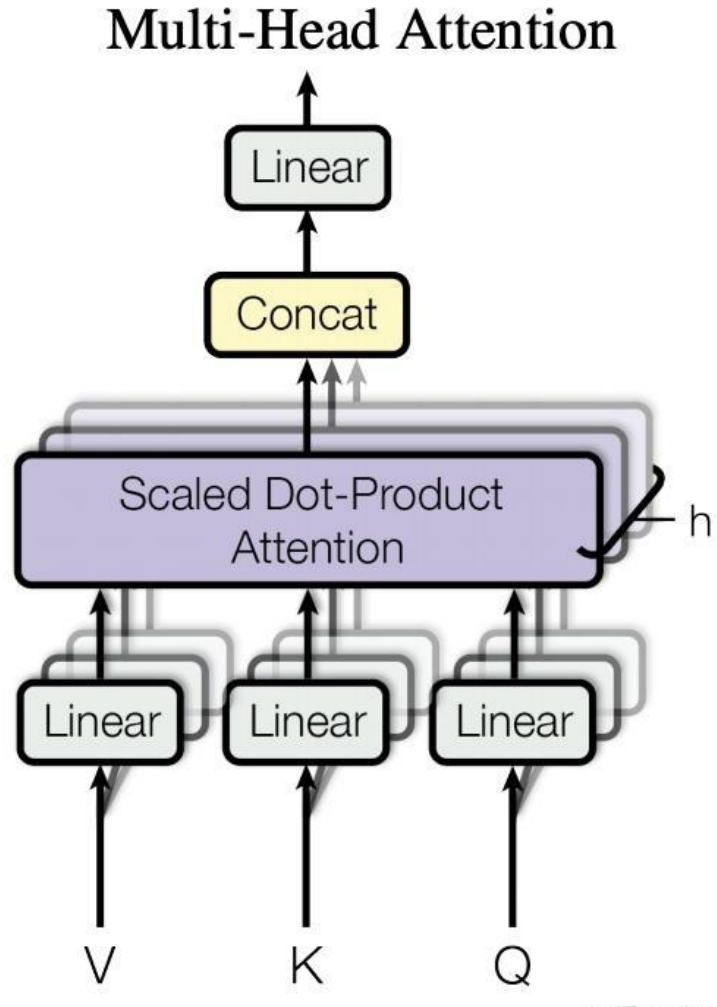
北京郵電大學 Beijing University of Posts and Telecommunications

```python
# 定义位置编码
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super().__init__()
        # 创建一个 [max_len, d_model] 的位置编码矩阵
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len).unsqueeze(1)  # [max_len, 1]
        div_term = torch.exp(torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model))
        # 奇数和偶数位置分别使用 sin 和 cos
        pe[:, 0::2] = torch.sin(position * div_term)  # 偶数位置
        pe[:, 1::2] = torch.cos(position * div_term)  # 奇数位置
        pe = pe.unsqueeze(0)  # 增加 batch 维度
        self.register_buffer('pe', pe)

    def forward(self, x):
        # x: [batch_size, seq_len, d_model]
        x = x + self.pe[:, :x.size(1)].to(x.device)
        return x
```

	i=0	i=1	i=2	i=4
POS=0 MASK	0	0	0	0
POS=1	sin(1 / power(10000,0 / 4))	cos(1 / power(10000,0 / 4))	sin(1 / power(10000,2 / 4))	cos(1 / power(10000,2 / 4))
POS=2	sin(2 / power(10000,0 / 4))	cos(2 / power(10000,0 / 4))	sin(2 / power(10000,2 / 4))	cos(2 / power(10000,2 / 4))
POS=3	sin(3 / power(10000,0 / 4))	cos(3 / power(10000,0 / 4))	sin(3 / power(10000,2 / 4))	cos(3 / power(10000,2 / 4))
POS=4	sin(4 / power(10000,0 / 4))	cos(4 / power(10000,0 / 4))	sin(4 / power(10000,2 / 4))	cos(4 / power(10000,2 / 4))
POS=5	sin(5 / power(10000,0 / 4))	cos(5 / power(10000,0 / 4))	sin(5 / power(10000,2 / 4))	cos(4 / power(10000,2 / 4))

Encoder的多头注意力(MHA)





注意力机制的代码解读


```python
class ScaledDotProductAttention(nn.Module):
    def __init__(self, d_k):
        super().__init__()
        self.scale = d_k ** -0.5  # 缩放因子

    def forward(self, Q, K, V, mask=None):
        """
        :param Q: [batch_size, heads, seq_len, d_k]
        :param K: [batch_size, heads, seq_len, d_k]
        :param V: [batch_size, heads, seq_len, d_v]
        :param mask: [batch_size, 1, 1, seq_len] 或 [batch_size, 1, seq_len, seq_len]
        :return: 注意力输出和注意力权重
        """
        scores = torch.matmul(Q, K.transpose(-2, -1)) * self.scale  # [batch_size, heads, seq_len, seq_len]
        if mask is not None:
            scores = scores.masked_fill(mask == 0, float('-inf'))
        attn = torch.softmax(scores, dim=-1)
        output = torch.matmul(attn, V)  # [batch_size, heads, seq_len, d_v]
        return output, attn
```

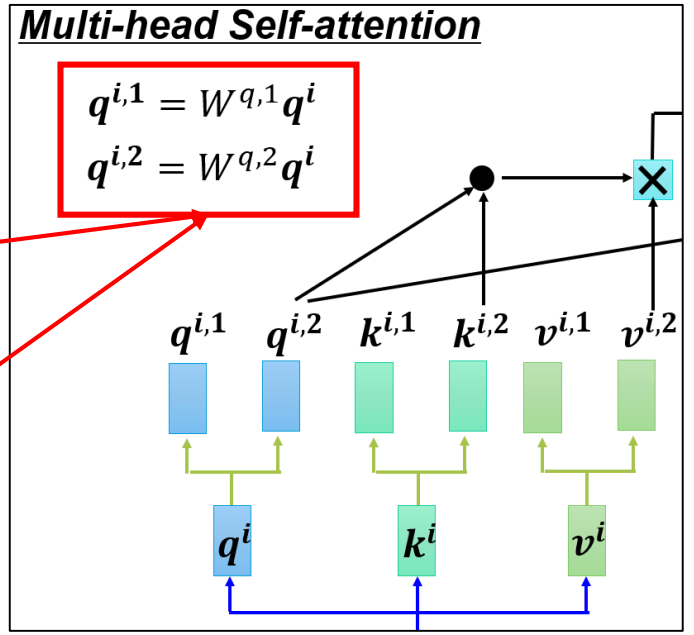
$$\text{Attention}(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

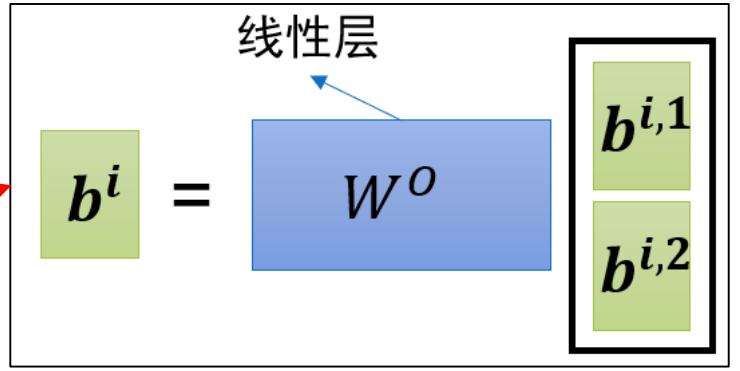
多头自注意力机制源码解读

北京郵電大學 Beijing University of Posts and Telecommunications

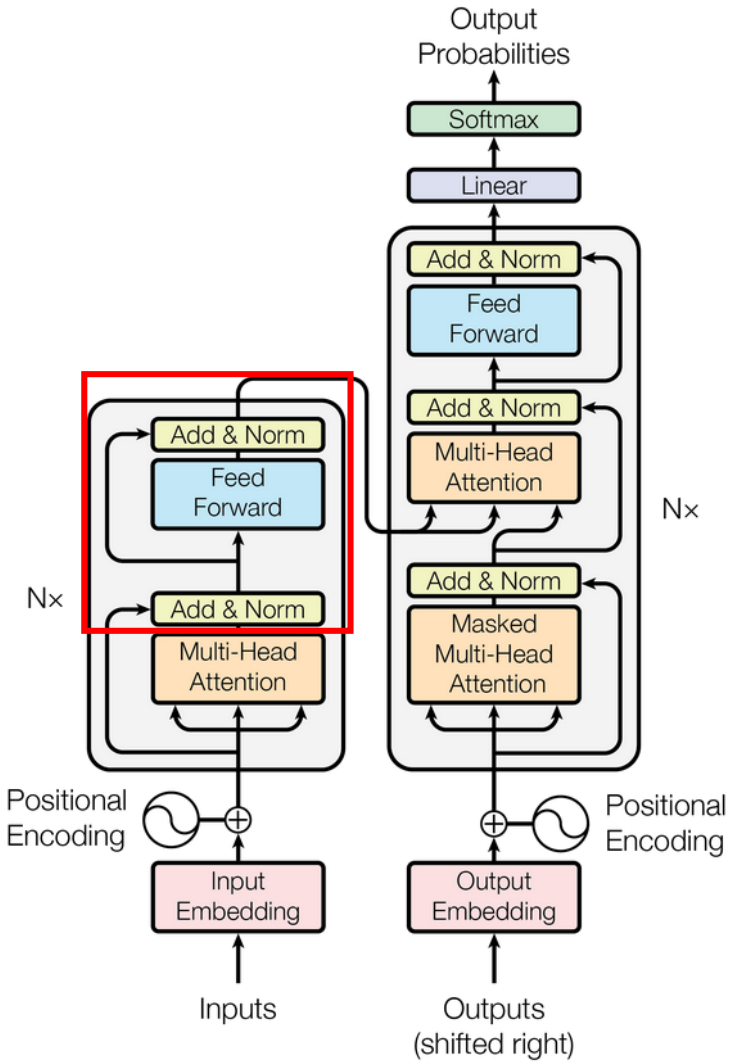
```python
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0, "d_model 必须能被 num_heads 整除"
        self.d_k = self.d_v = d_model // num_heads
        self.num_heads = num_heads

        # 定义线性变换层
        self.w_q = nn.Linear(d_model, d_model)
        self.w_k = nn.Linear(d_model, d_model)
        self.w_v = nn.Linear(d_model, d_model)
        self.fc = nn.Linear(d_model, d_model)

        self.attention = ScaledDotProductAttention(self.d_k)
        self.dropout = nn.Dropout(0.1)

    def forward(self, Q, K, V, mask=None):
        batch_size = Q.size(0)

        # 线性变换并分头
        Q = self.w_q(Q).view(batch_size, -1, self.num_heads, self.d_k).transpose(1,2)
        K = self.w_k(K).view(batch_size, -1, self.num_heads, self.d_k).transpose(1,2)
        V = self.w_v(V).view(batch_size, -1, self.num_heads, self.d_k).transpose(1,2)

        # 计算注意力
        if mask is not None:
            mask = mask.unsqueeze(1)  # 扩展维度以匹配多头
        output, attn = self.attention(Q, K, V, mask=mask)

        # 拼接多头的输出
        output = output.transpose(1,2).contiguous().view(batch_size, -1, self.num_heads * self.d_k)
        output = self.fc(output)
        return output
```

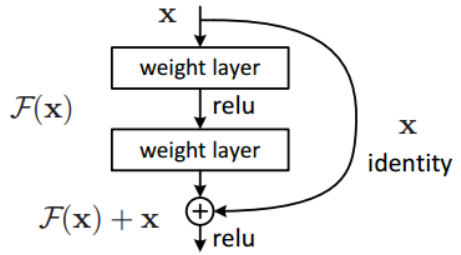
多头注意力机制划分多头的操作



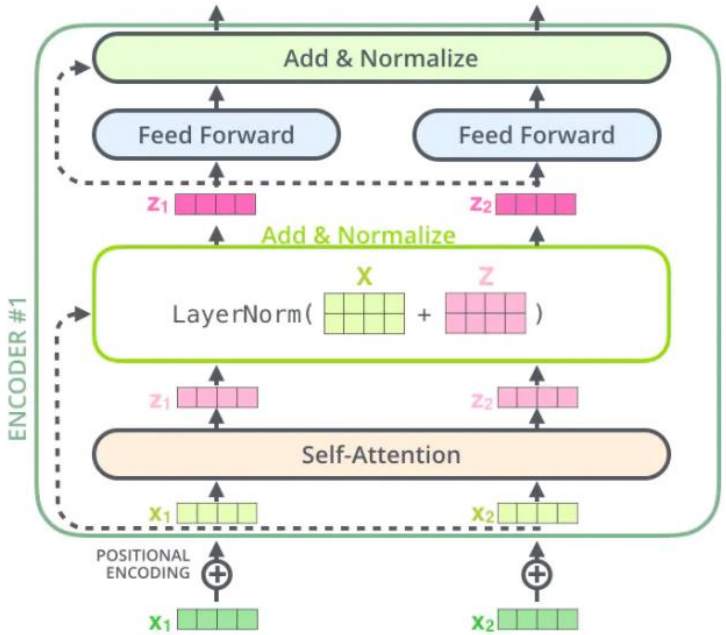


Transformer的Add&Norm





残差模块，防止梯度消失



$$LayerNorm(X+FFN(X))$$

$$FFN(x) = max(0, xW_1 + b1)W_2 + b2$$

$$LayerNorm(X+MHA(X))$$

Layer Normalization，即对矩阵的每一层进行标准化，首先计算出平均值和方差，然后计算矩阵中每一个数值的标准值。

（1）加快训练速度

（2）提高训练稳定性

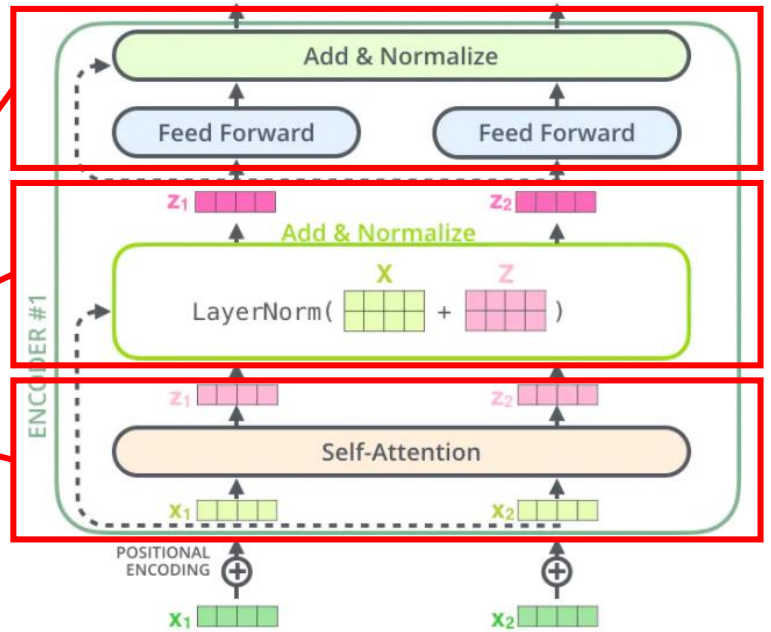
Add&Norm源码解读


```
230  class EncoderLayer(nn.Module):
231      def __init__(self, d_model, num_heads, d_ff, dropout):
232          super().__init__()
233          self.self_attn = MultiHeadAttention(d_model, num_heads)
234          self.ffn = PositionwiseFeedForward(d_model, d_ff, dropout)
235          self.norm1 = nn.LayerNorm(d_model)
236          self.norm2 = nn.LayerNorm(d_model)
237          self.dropout = nn.Dropout(dropout)
238
239      def forward(self, x, mask=None):
240          # 自注意力子层
241          attn_output = self.self_attn(x, x, x, mask)
242          x = x + self.dropout(attn_output)
243          x = self.norm1(x)
244          # 前馈神经网络子层
245          ffn_output = self.ffn(x)
246          x = x + self.dropout(ffn_output)
247          x = self.norm2(x)
248          return x
```

多头注意力计算

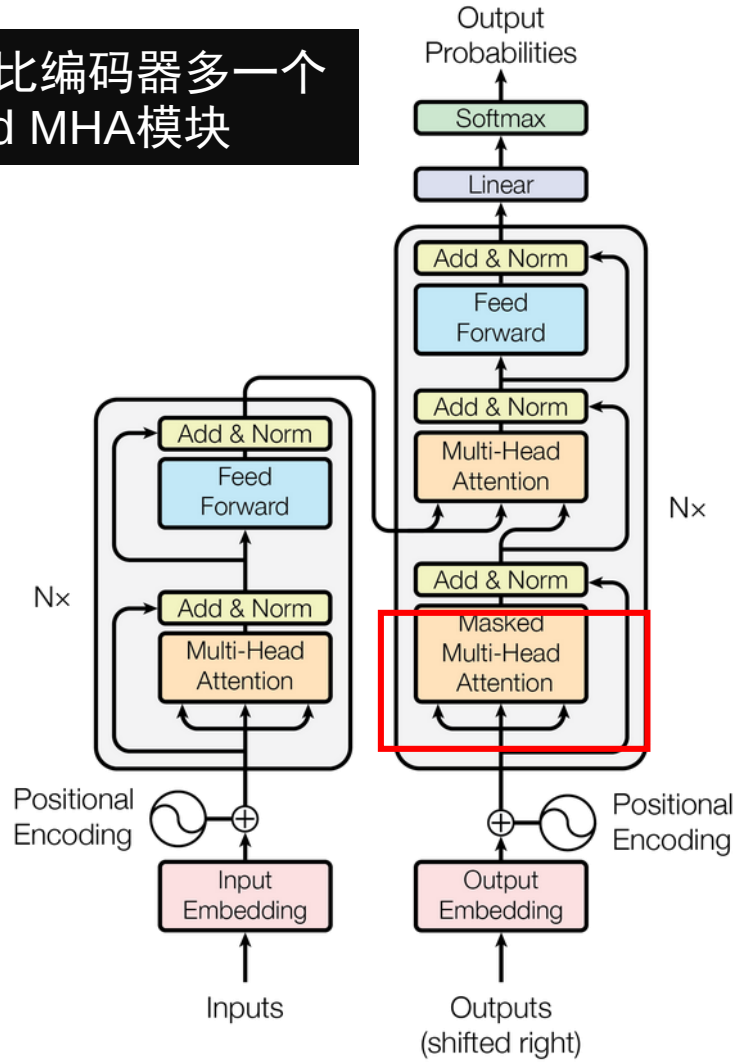
残差+归一化

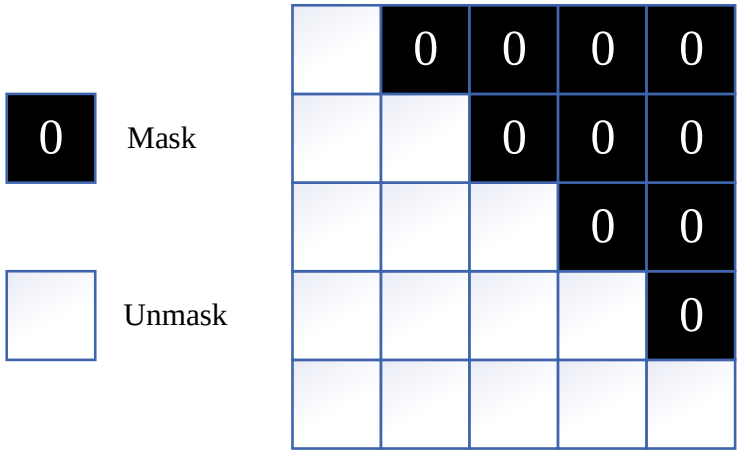
全连接+残差+归一化



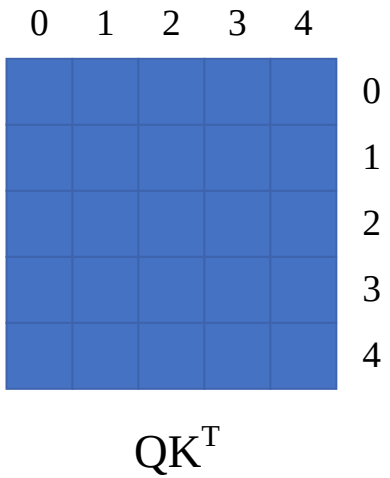
Transformer解码器中的Masked MHA

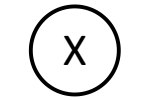
解码器比编码器多一个Masked MHA模块

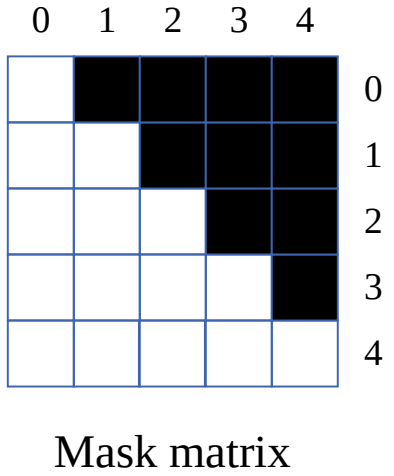


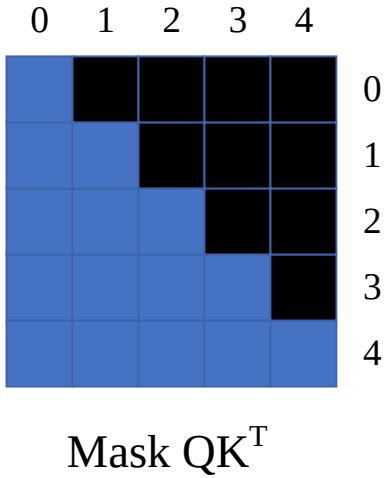


确保模型不会偷看“未来” 的输入







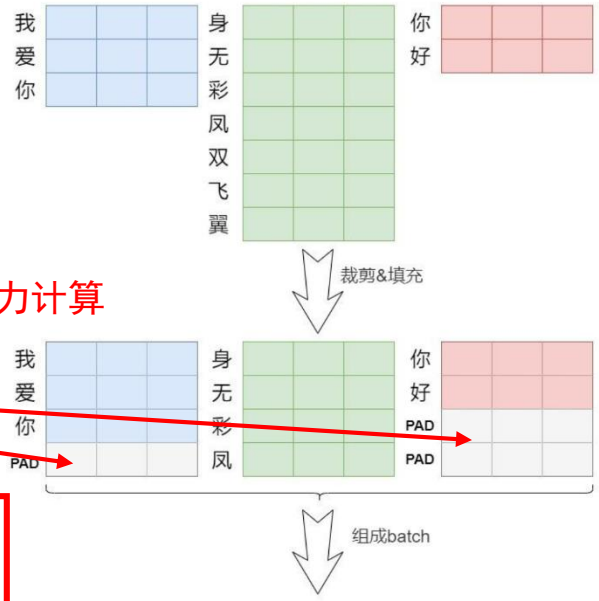


Masked MHA源码解读

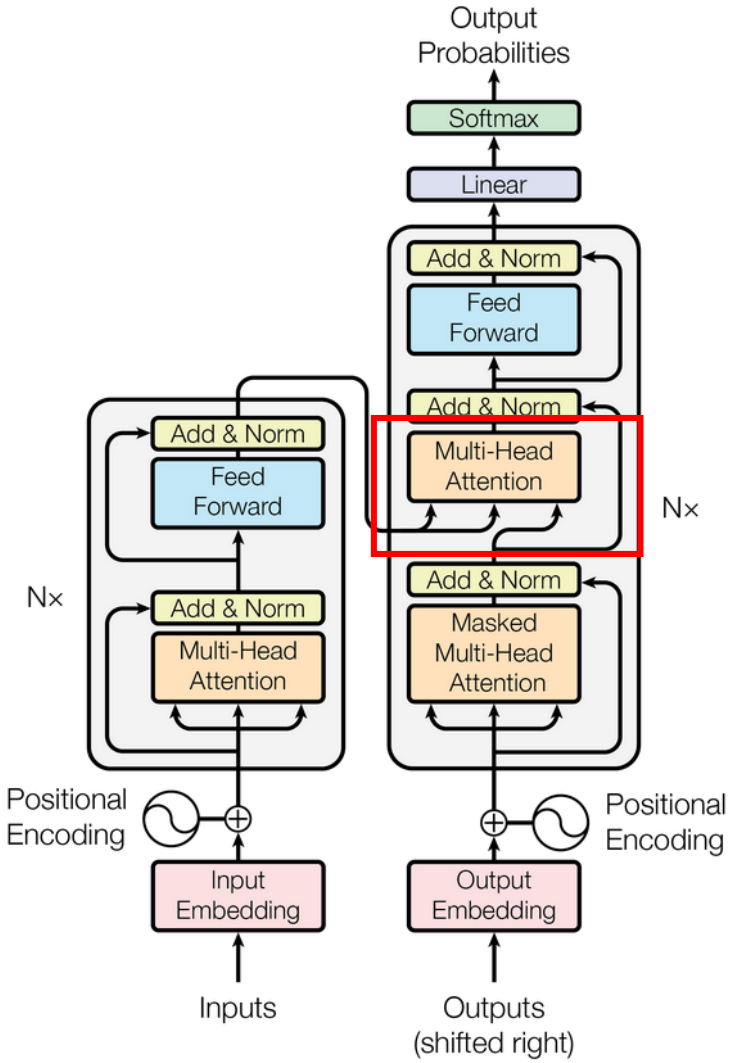

```python
class Transformer(nn.Module):
    def __init__(self, encoder, decoder):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder

    def make_src_mask(self, src):
        # 生成源序列的掩码，屏蔽填充位置
        src_mask = (src != en_vocab['<pad>']).unsqueeze(1).unsqueeze(2)
        return src_mask  # [batch_size, 1, 1, src_len]

    def make_trg_mask(self, trg):
        # 生成目标序列的掩码，包含填充位置和未来信息
        trg_pad_mask = (trg != zh_vocab['<pad>']).unsqueeze(1).unsqueeze(2)  # [batch_size, 1, 1, trg_len]
        trg_len = trg.size(1)
        trg_sub_mask = torch.tril(torch.ones((trg_len, trg_len), device=trg.device)).bool()  # [trg_len, trg_len]
        trg_mask = trg_pad_mask & trg_sub_mask  # [batch_size, 1, trg_len, trg_len]
        return trg_mask

    def forward(self, src, trg):
        src_mask = self.make_src_mask(src)
        trg_mask = self.make_trg_mask(trg)
        enc_output = self.encoder(src, src_mask)
        output = self.decoder(trg, enc_output, src_mask, trg_mask)
        return output
```

源域序列的填充位不需要注意力计算
要把其屏蔽

目标序列要屏蔽未来的信息



Multi-head Cross Attention



Masked MHA之后，目标词汇编码和源句子编码一起送入Cross MHA中，此处Q由目标词汇提供，K和V由源句子词汇提供，也就是信息从源句子词汇流向了目标词汇。

- K和V来源于Encoder最后一层的输出
- Q来源于Decoder中Masked MHA的输出

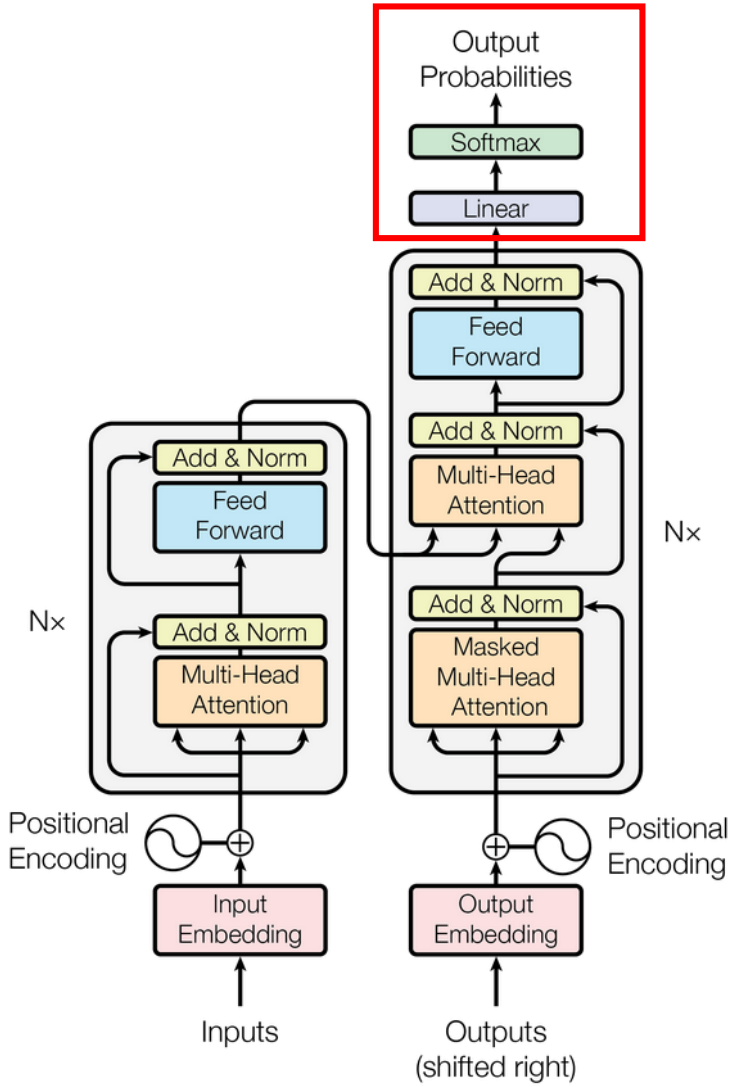
Multi-head Cross Attention源码解读


```
251  class DecoderLayer(nn.Module):
252      def __init__(self, d_model, num_heads, d_ff, dropout):
253          super().__init__()
254          self.self_attn = MultiHeadAttention(d_model, num_heads)
255          self.cross_attn = MultiHeadAttention(d_model, num_heads)
256          self.ffn = PositionwiseFeedForward(d_model, d_ff, dropout)
257          self.norm1 = nn.LayerNorm(d_model)
258          self.norm2 = nn.LayerNorm(d_model)
259          self.norm3 = nn.LayerNorm(d_model)
260          self.dropout = nn.Dropout(dropout)
261
262      def forward(self, x, enc_output, src_mask=None, trg_mask=None):
263          # 掩码多头自注意力子层
264          self_attn_output = self.self_attn(x, x, x, trg_mask)
265          x = x + self.dropout(self_attn_output)
266          x = self.norm1(x)
267          # 编码器-解码器注意力子层
268          cross_attn_output = self.cross_attn(x, enc_output, enc_output, src_mask)
269          x = x + self.dropout(cross_attn_output)
270          x = self.norm2(x)
271          # 前馈神经网络子层
272          ffn_output = self.ffn(x)
273          x = x + self.dropout(ffn_output)
274          x = self.norm3(x)
275          return x
```

Q来源于Decoder中Masked MHA的输出

K和V来源于Encoder最后一层的输出

Transformer的输出：经过Softmax



$$\text{Softmax}\left(\begin{array}{c} Z \\ \square \end{array} \times \begin{array}{c} W^{O} \\ \square \end{array}\right)$$

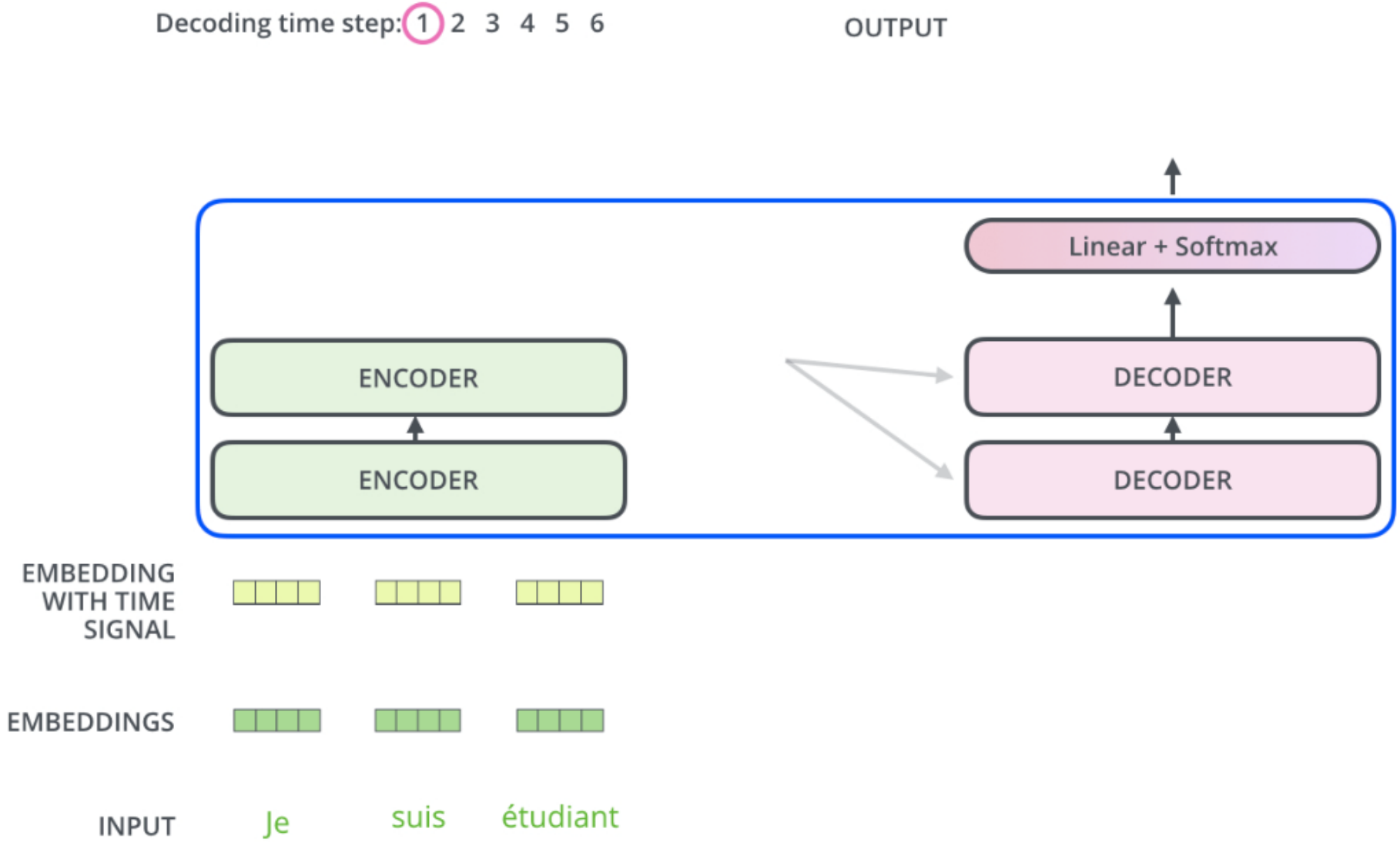
$$=$$

am	I	thanks	a student	<end>
0.1	0.8	0.1	0	0
0.9	0.02	0.01	0.03	0.02
0.05	0.01	0	0.85	0
0.02	0	0.05	0.03	0.9

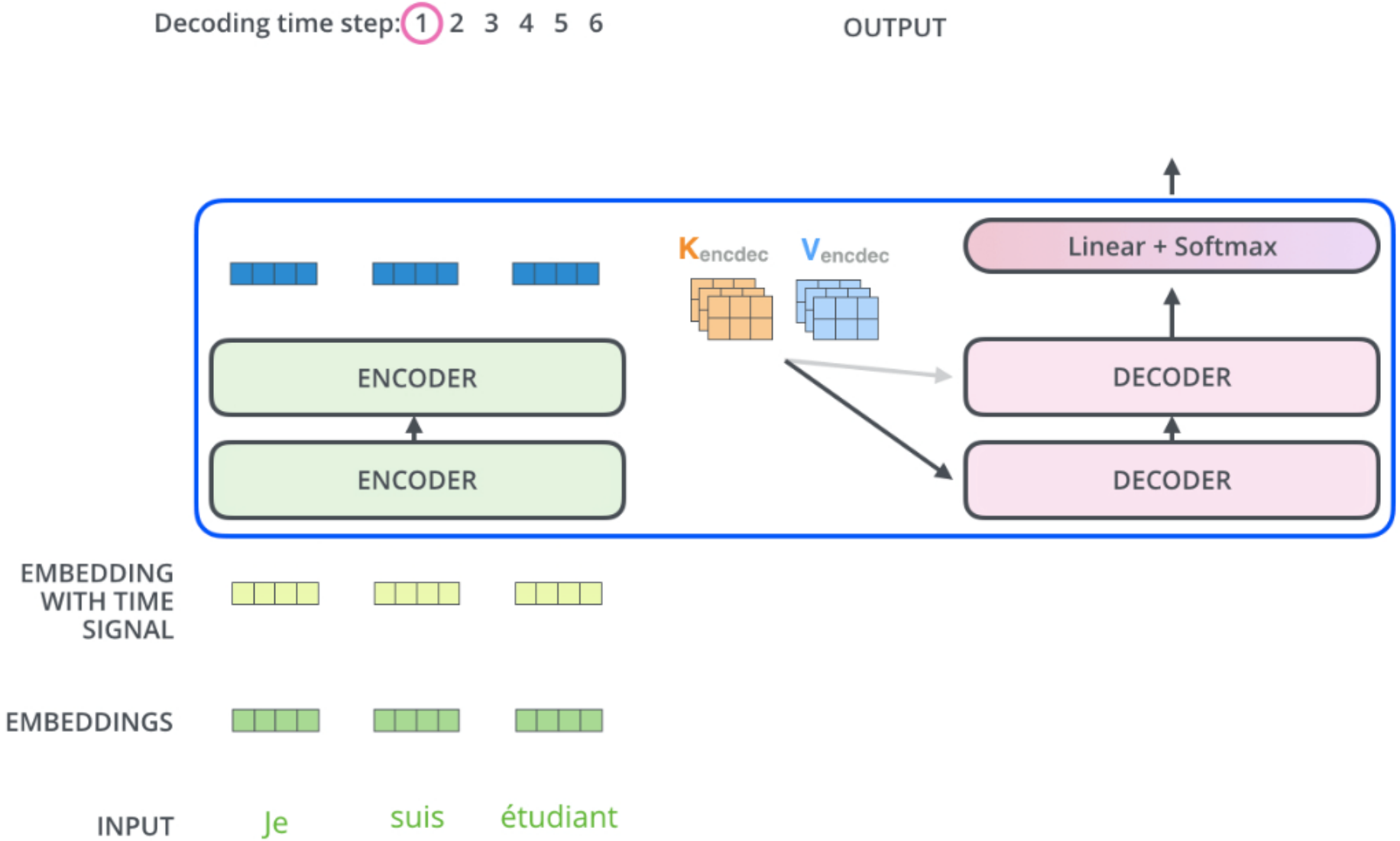
I
am
a student
<end>

输出

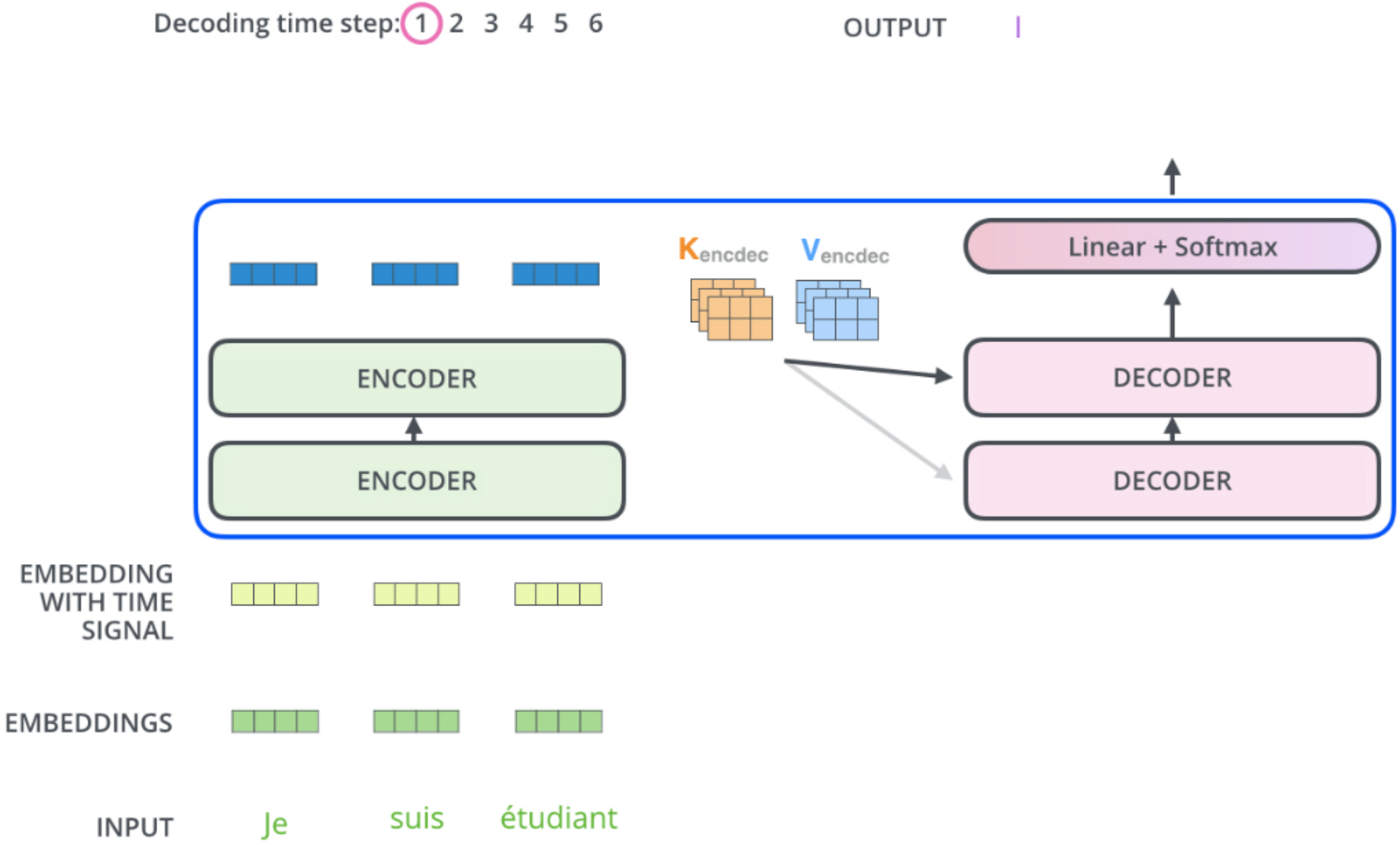
Transformer工作流程(机器翻译)



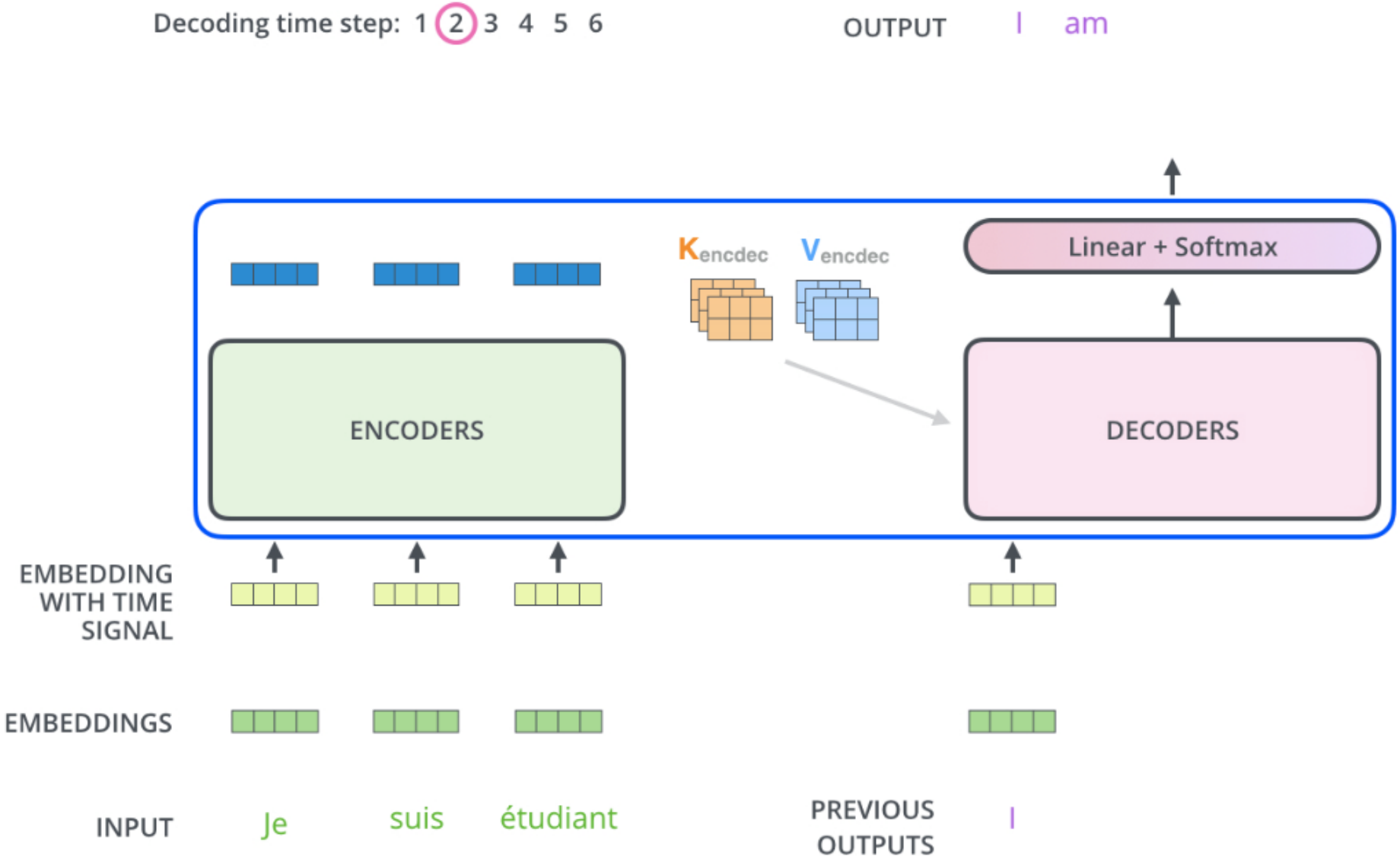
Transformer工作流程(机器翻译)



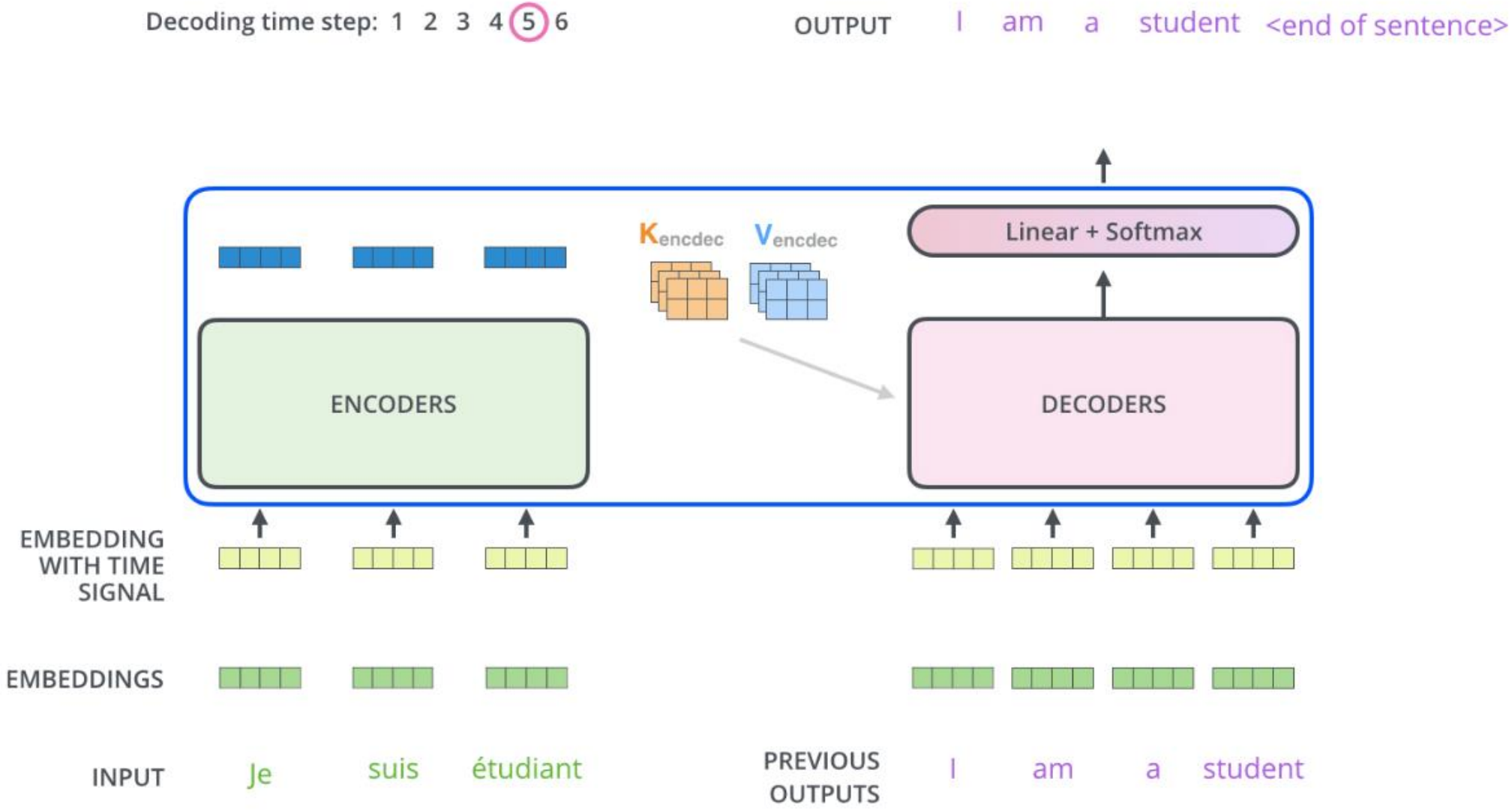
Transformer工作流程(机器翻译)



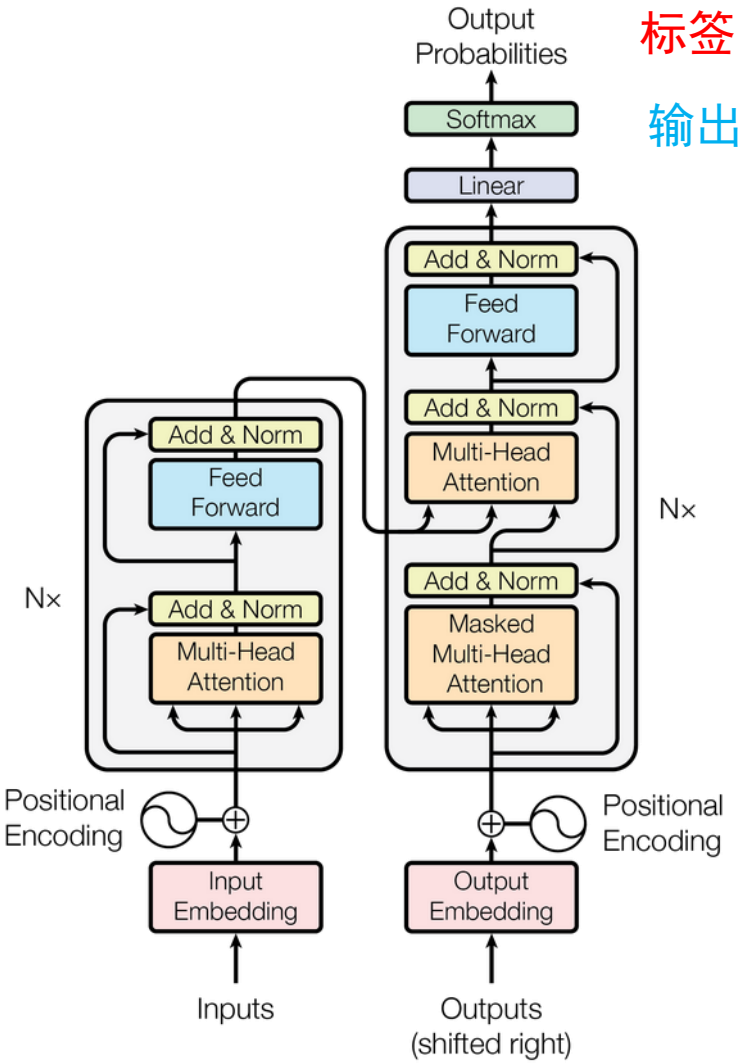
Transformer工作流程(机器翻译)



Transformer工作流程(机器翻译)



Transformer的训练

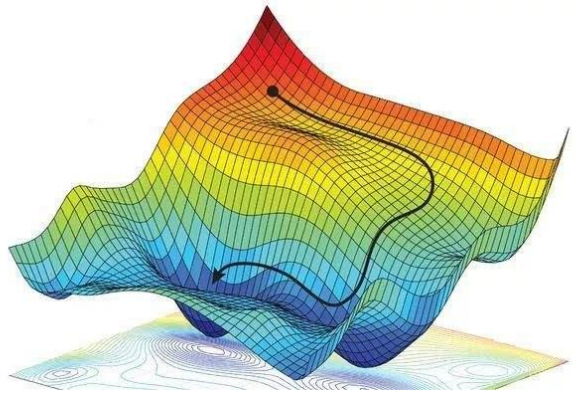


标签：I love eating pears ‘EOS’

输出：I love pears pears ‘EOS’

产生Loss
（通常是交叉熵损失）

反向传播
梯度下降更新网络权重



编码器输入：我爱吃梨

解码器输入：‘BOS’ I love eating pears

Transformer的优缺点总结

优点：

- 采用注意力机制捕捉序列数据中的长距离依赖关系
- 多头注意力机制可以在不同特征子空间同时捕捉不同的特征，提升表达力和准确性
- 训练不依赖序列的先后顺序，可以并行处理，提高训练速度和效率

缺点：

- 计算复杂度高，计算开销大（高算力）
- 训练数据需求量大，需要大量高质量标注数据（大数据）

作业练习

1. Transformer 模型中的自注意力机制（Self-attention）是如何捕捉长距离依赖关系的？请结合其计算过程说明原因。

2. 位置编码（Positional Encoding）在 Transformer 模型中的作用是什么？为什么不能直接用原始词向量而必须加入位置编码？请结合自注意力机制的特性说明。